에듀 체크

생성형 AI를 활용한 개인정보 유출 취약점 자동 진단 및 ISMS-P 인증기준 확인

팀원 : 박경민(팀장), 송예린, 어승효, 윤경은, 장은빈, 홍용민, 최재용

목차

1. 프로젝트 개요
2. 프로젝트 팀 구성 및 역할
3. 프로젝트 수행 절차 및 방법
4. 프로젝트 수행 경과
5. 자체 평가

에듀체크

01

생성형 AI를 활용한 개인정보 유출 취약점 자동 진단 및 ISMS-P 인증기준 확인

프로젝트 개요

01 프로젝트 개요 – 주제 및 선정 배경, 기획의도

프로젝트 주제

생성형 AI를 활용한 개인정보 유출 취약점 자동 진단 및 ISMS-P 인증기준 확인

선정 배경

- 2026년 9월, 온라인 교육 사이트 ‘맑은 소프트’, ‘라이브 클래스’등에서 개인정보 유출이 확인됨
- 교육업계는 학생·학부모 정보를 함께 보관해, 금융정보·주소 등 민감정보의 추가 피해 우려가 큼
- ISMS-P가 취약점 확인·개선을 요구하지만, 필수 인증 대상이 아닌 소규모 사업자는 취약한 경우가 많음

기획 의도

- 개인정보 유출로 이어질 수 있는 웹 취약점을 자동으로 진단해 유출 사고를 줄임
- 진단 결과를 주요정보통신기반시설·ISMS-P 기준과 개인정보 보호법 조문에 연결
- 위험성과 개선 방안을 보고서로 제공해, 점검 결과가 바로 조치로 이어지게 함

01 프로젝트 개요 – 특화 포인트와 차별점

범용 도구를 대체하는 것이 아니라, 국내 기준에 맞춘 결과물을 한 번에 만드는 데 초점

구분	범용 보안 스캐너	AI 자율 모의해킹 도구	에듀 체크
점검 기준	OWASP 등 범용 기준	범용 공격 시나리오	**KISA 가이드의 개인정보 유출 관련 웹 취약점 5개 영역**
판정 방식	규칙·시그니처 기반	AI 에이전트가 직접 공격 시도	**대부분 규칙 기반 판정, AI는 법령 매핑과 보고서 문장에 활용**
결과물	취약점 목록	공격 경로와 재현 결과	**취약점 + ISMS-P·법령 연결 + 경영진용·실무진용 보고서**

※ 도구 유형별 일반적 특징을 비교한 것으로, 개별 제품의 성능 비교가 아님

01 프로젝트 개요 – 프로젝트 내용

구현 내용

- 학생·강사의 개인정보와 성적을 다루는 교육 관리 웹 서비스 구축
- 정보 누출, 약한 비밀번호 정책, 취약한 비밀번호 복구 절차, 불충분한 인증 절차, 불충분한 권한 검증 5개 영역의 자동 진단 도구
- 진단 결과를 ISMS-P 인증기준과 개인정보 보호법 조문에 매핑
- 경영진용·실무진용 PDF 보고서 생성

컨셉

진단 → 법적 영향 → 보고서

점검 결과를 법적 영향과 조치 방안까지 한 번에 연결한다.

01 프로젝트 개요 – 활용 장비 및 재료

구분	개발환경 · 기술 스택 · 자료
개발 환경	Python 3.12, Git·GitHub 버전 관리 (Windows, macOS)
대상 웹 서비스	Flask 3.1, Flask-SQLAlchemy(SQLite), Flask-Login, Jinja2
취약점 진단	Python requests, BeautifulSoup (HTTP 응답, 인증 상태, 사용자 권한 분석)
AI · 법령 분석	OpenAI API(GPT, 임베딩), ChromaDB(유사 사례 검색), Pydantic
보고서 생성	Jinja2 템플릿, WeasyPrint (PDF 변환)
기준 자료	KISA 2026 상세가이드(웹 21개 항목), ISMS-P 인증기준 안내서, 개인정보 보호법, 안전성 확보조치 기준 고시, 침해 사례 수집 시트

01 프로젝트 개요 – 프로젝트 구조

STEP 1

대상 서비스

- 에듀 체크 교육 관리 플랫폼
- 학생·강사 계정
- 개인정보, 성적, 문의 게시판

STEP 2

자동 진단 도구

- URL 입력 후 5개 영역 점검
- Rule 기반 판정
- Evidence 수집

STEP 3

법령·ISMS-P 매핑

- ISMS-P 인증기준 선별
- 개인정보 보호법 조문 연결
- 생성형 AI가 해당 기준·조문을 고르고 이유 작성

STEP 4

보고서 생성

- 경영진용 요약 PDF
- 실무진용 상세 PDF
- 위험도, 개선 방안, 유사 사례

점검 5개 영역

정보 누출 | 약한 비밀번호 정책 | 취약한 비밀번호 복구 절차 | 불충분한 인증 절차 | 불충분한 권한 검증

01 프로젝트 개요 – 활용방안 및 기대효과

활용방안

- 보안 인력이 부족한 소규모 교육·서비스 사업자의 자가 점검
- 서비스 개발·배포 전 정기 취약점 점검 자동화
- ISMS-P 인증 준비 시 사전 점검 자료
- 경영진 보고용 요약본과 실무 조치용 상세본을 함께 확보

기대 효과

- 취약점을 자동으로 진단해 보안 인력의 업무 부담 감소
- 개인정보 유출을 예방해 사생활 보호
- 향후 ISMS-P 인증 준비와 연계 가능

점검부터 법적 영향, 조치 방안까지 한 번에 확인해 보안 인력이 없는 서비스도 스스로 점검·개선할 수 있다

※ 현재 점검 대상: 내부망(localhost 등)에서 실행 중인 자체 구축 교육 플랫폼

02

에듀체크

생성형 AI를 활용한 개인정보 유출 취약점 자동 진단 및 ISMS-P 인증기준 확인

프로젝트 팀 구성 및 역할

02 프로젝트 팀 구성 및 역할

박경민	불충분한 권한검증 진단 도구 구현, 발표
송예린	보고서 모듈(엔진, 지식베이스) 구현, 발표
어승효	불충분한 인증 절차 진단 모듈 구현, Git 브랜치 통합 및 형상 관리
윤경은	취약점 진단 통합 실행 모듈 구현
장은빈	비밀번호 재설정 기능 구현, 취약한 비밀번호 복구 절차 진단 도구 구현
최재용	법령 매핑 모듈 (취약점 → ISMS-P 인증기준 · 개인정보 보호법 · 안전성 확보조치 기준 조문·항)
홍용민	대상 서비스 구현, 정보 누출/약한 비밀번호 진단 도구 구현

에듀체크

03

생성형 AI를 활용한 개인정보 유출 취약점 자동 진단 및 ISMS-P 인증기준 확인

프로젝트 수행 절차 및 방법

03 프로젝트 수행 절차 및 방법

구분	기간	활동
사전 기획	10/1(목)	- 프로젝트 기획 및 주제 선정 - 협업, 개발환경 구축
1차 개발	10/2(금)	- 플랫폼 개발 - 취약점 진단 모듈 개발 - 법령 매핑 엔진 구현
2차 개발	10/6(화)	- 취약점 진단 모듈과 법령 매핑 엔진을 결합 - 보고서 출력 및 결과 분석 - 발표자료 준비
최종 발표	10/7(수)	- 최종 보고서 작성 및 발표

✓ 총 개발 기간 : 10/1(목) ~ 10/2(수), 10/6~10/7 (총 4일)

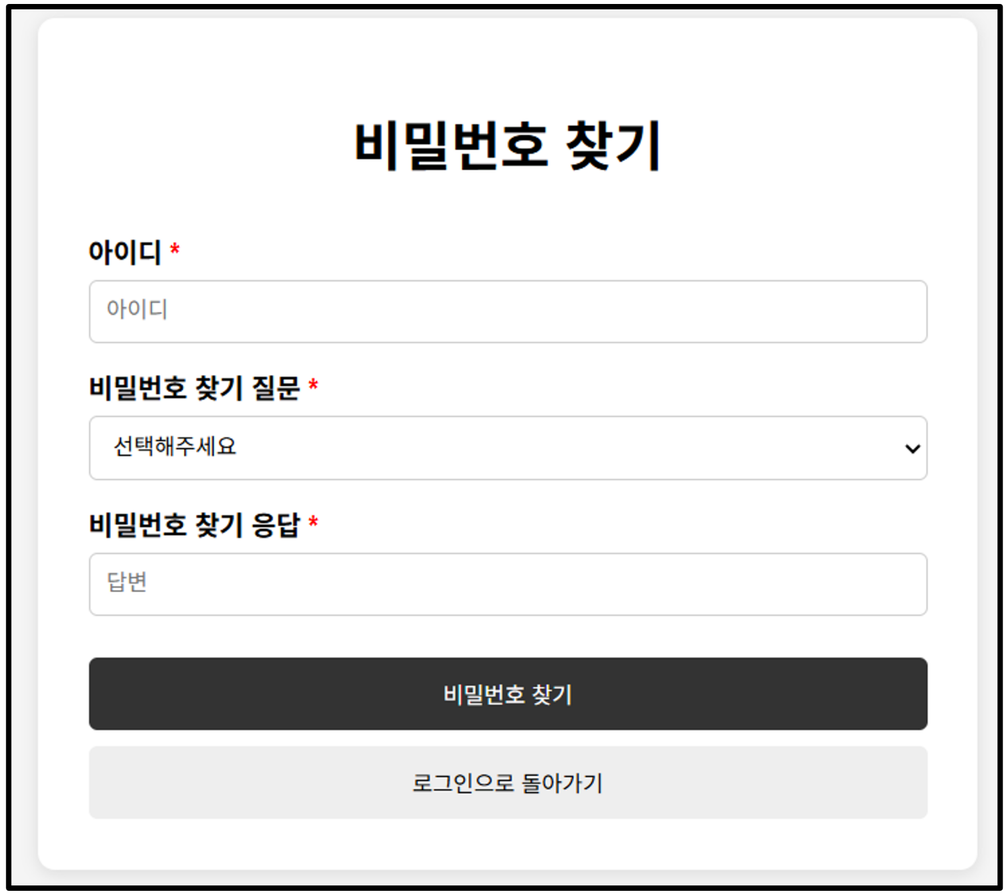
에듀체크

04

생성형 AI를 활용한 개인정보 유출 취약점 자동 진단 및 ISMS-P 인증기준 확인

프로젝트 수행 경과

04 취약점 분석 및 시나리오 - 취약한 비밀번호 복구 절차



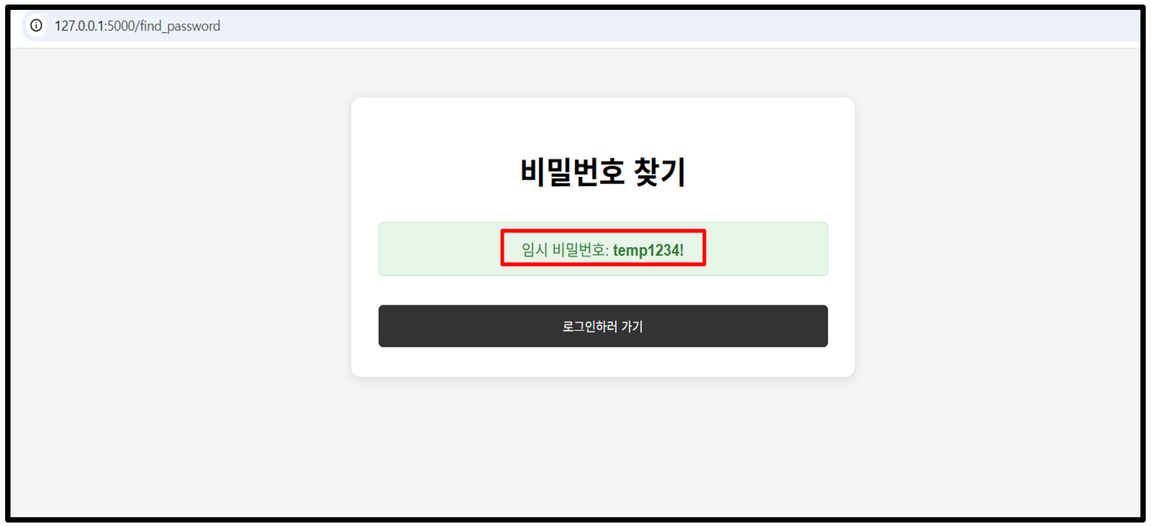
취약한 비밀번호 복구 절차 취약점 진단

- 진단 대상 : **/find_password**
- 확인 정보 : 사용자 계정(username), 질의응답 기반 임시 비밀번호
- 비밀번호 찾기 시 외부 채널 발송 등 추가 인증 없음
- 진단 목표 : **화면상에 임시 비밀번호가 평문으로 즉시 노출되는지 검증**

핵심 포인트

- 별도 외부 인증 채널 발송 없이 웹 화면 내 임시 비밀번호 즉시 노출

04 취약점 분석 및 시나리오 - 취약한 비밀번호 복구 절차



점검 항목 - 1. 웹 화면 내 임시 비밀번호 노출

- 비밀번호 찾기 요청 시 안전한 외부 채널이 아닌 웹 브라우저 화면에 임시 비밀번호가 평문으로 노출됨을 식별
- 유추하기 쉬운 고정 패턴으로 임시 비밀번호가 발급됨으로 취약

04 취약점 분석 및 시나리오 - 취약한 비밀번호 복구 절차

```python
# ------------------------------------------------
# 비밀번호 재설정
# ------------------------------------------------

@app.route('/find_password', methods=['GET', 'POST'])
def find_password():
    error = None
    found_password = None

    if request.method == 'POST':
        username = request.form.get('username')

        # 1. 사용자 존재 여부만 확인하거나 student1 계정이면 즉시 임시 비밀번호 발급
        user = User.query.filter_by(username=username).first()

        if user:
            # 2. 임시 비밀번호 발급
            new_password = "temp1234!"
            user.set_password(new_password)
            db.session.commit()

            found_password = new_password
        else:
            error = "입력하신 사용자 정보를 찾을 수 없습니다."

    return render_template('find_password.html', error=error, found_password=found_password)
```

점검 항목 - 2. 고정 패스워드 생성

- 난수 생성 로직 없이 고정된 문자열(Temp1234!)로 비밀번호 갱신
- 갱신된 임시 비밀번호를 평문 변수(found_password)에 그대로 할당

04 취약점 분석 및 시나리오 - 취약한 비밀번호 복구 절차

```
{% if found_password %}
    <div class="success-box">
        임시 비밀번호: <strong>{{ found_password }}</strong>
    </div>
    <button class="submit-button" onclick="location.href='{{ url_for('login') }}'">
        로그인하러 가기
    </button>
{% else %}
```

점검 항목 - 3. 웹 템플릿 내 평문 노출

- 서버 변수 found_password 값을 마스킹 없이 그대로 출력
- 비밀번호 복구 성공 시 .success-box 영역 내 평문 노출 확인
- 외부 링크나 인증 절차 없이 '로그인하러 가기' 버튼만 제공

04 취약점 분석 및 시나리오 - 취약한 비밀번호 복구 절차

수동 한계 & 자동화 파이프라인

수동 진단 절차

- /find_password 페이지 접근 및 사용자 질의응답 입력
- 비밀번호 찾기 요청 후 반환된 웹 응답 페이지 직접 확인
- HTML 본문 및 화면상 임시 비밀번호 노출 확인

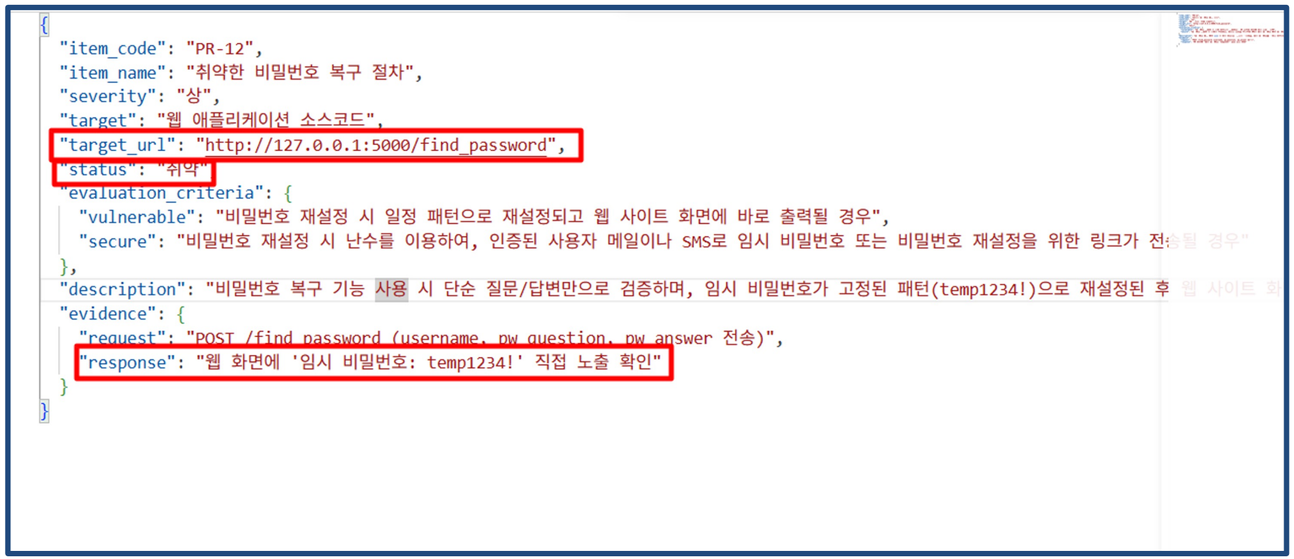
한계점

- 고정 패턴 및 난수성 검증을 위한 반복 요청•검증의 한계
- 진단 증적(요청/응답 패킷) 캡처 누락 및 인적 실수 발생

자동 진단 파이프라인



04 취약점 분석 및 시나리오 - 취약한 비밀번호 복구 절차



```
{
  "item_code": "PR-12",
  "item_name": "취약한 비밀번호 복구 절차",
  "severity": "상",
  "target": "웹 애플리케이션 소스코드",
  "target_url": "http://127.0.0.1:5000/find_password",
  "status": "취약",
  "evaluation_criteria": {
    "vulnerable": "비밀번호 재설정 시 일정 패턴으로 재설정되고 웹 사이트 화면에 바로 출력될 경우",
    "secure": "비밀번호 재설정 시 난수를 이용하여, 인증된 사용자 메일이나 SMS로 임시 비밀번호 또는 비밀번호 재설정을 위한 링크가 전
  },
  "description": "비밀번호 복구 기능 사용 시 단순 질문/답변만으로 검증하며, 임시 비밀번호가 고정된 패턴(temp1234!)으로 재설정된 후
  "evidence": {
    "request": "POST /find_password (username, pw_question, pw_answer 전송)",
    "response": "웹 화면에 '임시 비밀번호: temp1234!' 직접 노출 확인"
  }
}
```

점검 항목 - 4. 스캐너 진단 판정(취약/양호 기준)

- 응답 본문 내 '임시 비밀번호 : temp1234! 평문 패턴 식별 → 취약 판정
- 난수 기반 외부 채널 전송 및 화면 노출 차단 기준 미충족

  → 위험도 상 확인 가능

04 취약점 분석 및 시나리오 - 불충분한 인증 절차

불충분한 인증 절차 취약점 개요



불충분한 인증 절차 취약점 진단

- 진단 대상: `/student/profile`
- 변경 가능 개인정보: 이름 / 생년월일 / 전화번호
- 로그인 후 개인정보 변경 시 현재 비밀번호·OTP 등 추가 재인증 없음
- 진단 목표: **재인증 없이 실제 개인정보 변경이 가능한지 검증**

핵심 포인트

현재 비밀번호·OTP 등 별도 재인증 없이

개인정보 수정 가능

04 취약점 분석 및 시나리오 - 불충분한 인증 절차

수동 진단 절차와 자동화 필요성

수동 진단

로그인

→ 기존 개인정보 확인

→ 재인증 절차 확인

→ 전화번호 변경

→ 실제 변경 여부 확인

→ 원본값 복구

자동화 필요성

수동 진단의 한계

- 반복 작업 발생
- 변경 후 검증 누락 가능
- 테스트 데이터 원복 누락 가능
- 진단자마다 판정 기준 차이
- 다수 대상 반복 진단 시 시간 소요

수동 진단의 검증 절차를
HTTP 기반 동적 진단으로 자동화

04 취약점 분석 및 시나리오 - 불충분한 인증 절차

자동 진단 파이프라인

자동 진단 파이프라인

Target URL
↓
학생 계정 로그인
↓
인증 상태 검증
↓
개인정보 수정 페이지 접근
↓
기존 개인정보 수집
↓
재인증 필드 탐지
↓
테스트용 전화번호 생성
↓
재인증 없이 변경 요청
↓
개인정보 재조회
↓
실제 변경 여부 비교
↓
vulnerable / pass / unknown
↓
원본 개인정보 복구
↓
복구 결과 검증

동적 진단

- HTML 구조만으로 취약 여부를 분석하지 않음
- 실제 로그인 Session 사용
- 실제 HTTP POST 요청
- 변경 후 서버 값 재조회

04 취약점 분석 및 시나리오 - 불충분한 인증 절차

구현 아키텍처 및 주요 함수

함수	역할
`_login_and_verify()`	로그인 및 실제 인증 상태 확인
`_fetch_profile()`	개인정보 수정 페이지 조회
`_parse_profile_form()`	개인정보/재인증 필드 추출
`_submit_profile()`	변경 요청 전송
`_build_result()`	결과 구조 생성
`_normalize_base_url()`	Target URL 형식 검증

외부 진입 함수 ※ 테스트 계정 기본값

```python
def check_insufficient_auth(
    url: str,
    username: str = "student1",
    password: str = "student123",
) -> Dict[str, object]:
```

내부 함수 구조

check_insufficient_auth()
↓
URL 확인
_normalize_base_url()
↓
로그인 및 인증 확인
_login_and_verify()
↓
개인정보 페이지 조회
_fetch_profile()
└ _parse_profile_form()
↓
개인정보 변경 요청
_submit_profile()
↓
변경 결과 비교
phone_changed
other_values_preserved
↓
결과 반환
_build_result()
↓
원본 개인정보 복구
finally

04 취약점 분석 및 시나리오 - 불충분한 인증 절차

핵심 판정 로직 및 오탐 방지

```
phone_changed = changed_values["phone"] == test_values["phone"]
other_values_preserved = all(
    changed_values[field] == original[field]
    for field in ("name", "birth_date")
)
```

```
if phone_changed and other_values_preserved:
    result = "vulnerable"
```

재인증 입력창이 없다는 사실만으로
취약 판정하지 않음

판정 기준

Vulnerable

재인증 정보 미제공

+

개인정보 변경 요청

+

서버에 실제 변경 반영

Unknown

로그인 실패

네트워크 오류

응답 불명확

판정 근거 부족

Pass

재인증 절차 존재

+

재인증 없는 요청 거부

+

원본 개인정보 유지

04 취약점 분석 및 시나리오 - 불충분한 인증 절차

테스트 데이터 보호 및 결과 구조

자동 원복

기존 개인정보 저장
↓
테스트 값 변경
↓
진단 수행
↓
finally
↓
원본값 다시 전송
↓
재조회
↓
복구 여부 확인

모듈 반환 결과 구조

```
{
  "path": "/student/profile",
  "content": [...],
  "category": "불충분한 인증 절차",
  "result": "vulnerable",
  "severity": "high",
  "reason": "..."
}
```

통합 Scanner 및 AI 분석/보고서 모듈에서
활용할 수 있도록 구조화된 결과 반환

04 취약점 분석 및 시나리오 - 불충분한 인증 절차

AI 연계 구조

AI 연계 구조

실제 HTTP 요청

↓

Evidence 수집

↓

Rule 기반 취약점 판정

↓

구조화된 결과

↓

생성형 AI

↓

취약점 설명 / ISMS-P 연계 /
보고서 생성

AI 활용 방식

AI는 자동 진단기가 확보한 객관적인 증거와
판정 결과를 기반으로
설명 및 보고서 생성을 수행

취약 여부 자체를 생성형 AI가
판단하지 않도록 설계

04 취약점 분석 및 시나리오 - 불충분한 인증 절차

통합 테스트 결과

```
(.venv) C:\SK34\34_module_proj2\vuln_assessment\scanner>python insufficient_auth.py
{
  "path": "/student/profile",
  "content": [
    "로그인 및 인증 페이지 접근 성공 (HTTP 302 -> 302 -> 200, 최종 URL http://localhost:5000/student; HTTP 200, 최종 URL http://localhost:5000/student/mypage)",

    "기존 개인정보(name, birth_date, phone) 수집 성공 (HTTP 200, 최종 URL http://localhost:5000/student/profile)",
    "재인증 값을 제공하지 않은 개인정보 변경 요청: HTTP 302 -> 200, 최종 URL http://localhost:5000/student/mypage, 폼의 재인증 필드=없음",
    "변경 후 재조회 결과: 전화번호 변경=확인, 이름/생년월일 유지=확인",
    "원본 개인정보 복구/유지 성공 (HTTP 302 -> 200, 최종 URL http://localhost:5000/student/mypage)"
  ],
  "category": "불충분한 인증 절차",
  "result": "vulnerable",
  "severity": "high",
  "reason": "현재 비밀번호나 OTP 등 재인증 정보를 전송하지 않았는데도 서버에서 전화번호 변경이 실제 반영되었습니다."
}
```

결과 해석

Category - 불충분한 인증 절차

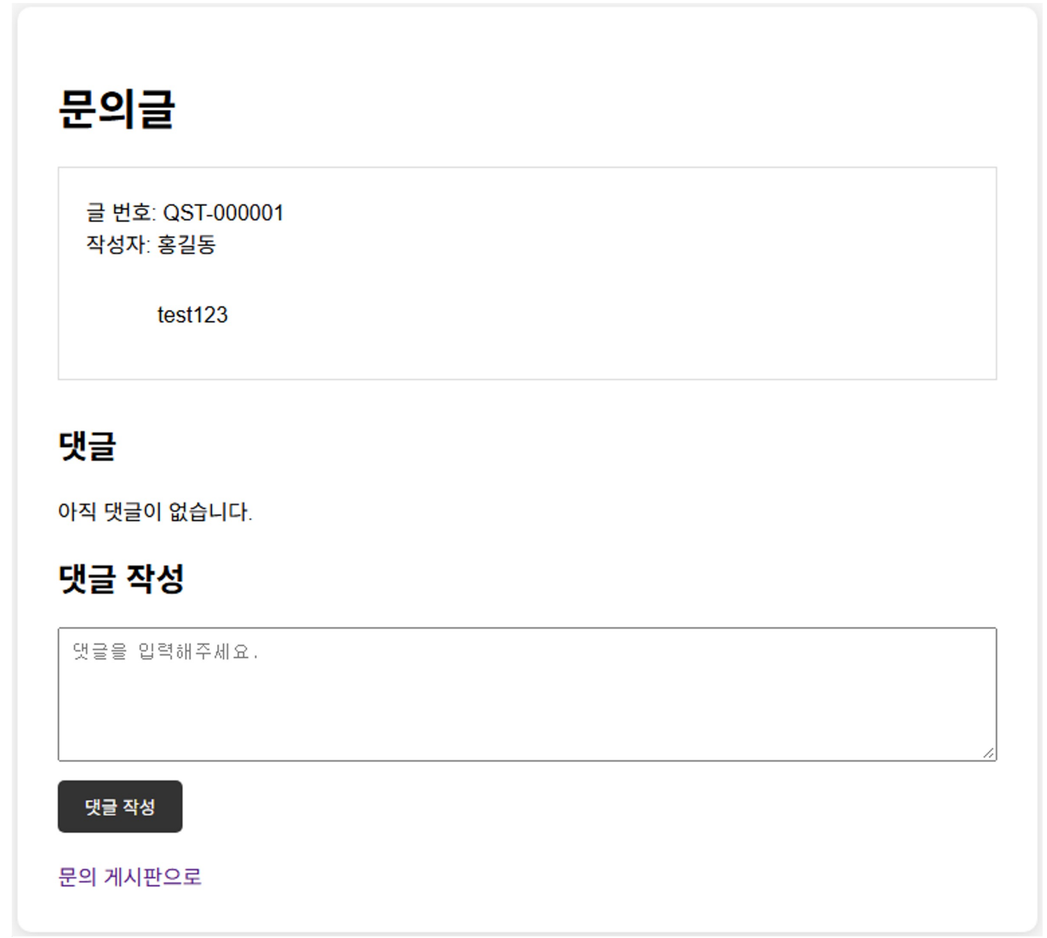
Result - VULNERABLE

Severity - HIGH

체크 포인트

✓ 로그인 성공
✓ 기존 개인정보 수집
✓ 재인증 없이 변경 요청
✓ 전화번호 실제 변경 확인
✓ 이름/생년월일 유지 확인
✓ 원본 개인정보 복구 성공

04 취약점 분석 및 시나리오 - 불충분한 권한 검증



가이드 항목

주요정보통신기반시설 취약점 - 불충분한 권한 검증(위험도 : 상)

판단 기준

- **취약**
  중요 페이지에서 사용자 검증 로직이 미흡하여, 타 사용자의 권한 탈취가 가능한 경우
- **양호**
  중요 페이지에서 사용자 검증 로직이 구현되어 있어, 타 사용자의 권한 탈취가 제한된 경우

※ **중요 페이지** : 로그인 후 접근하는 학생, 강사 전용 페이지, 타인의 문의 글

04 취약점 분석 및 시나리오 - 불충분한 권한 검증

수동진단 절차 및 한계점

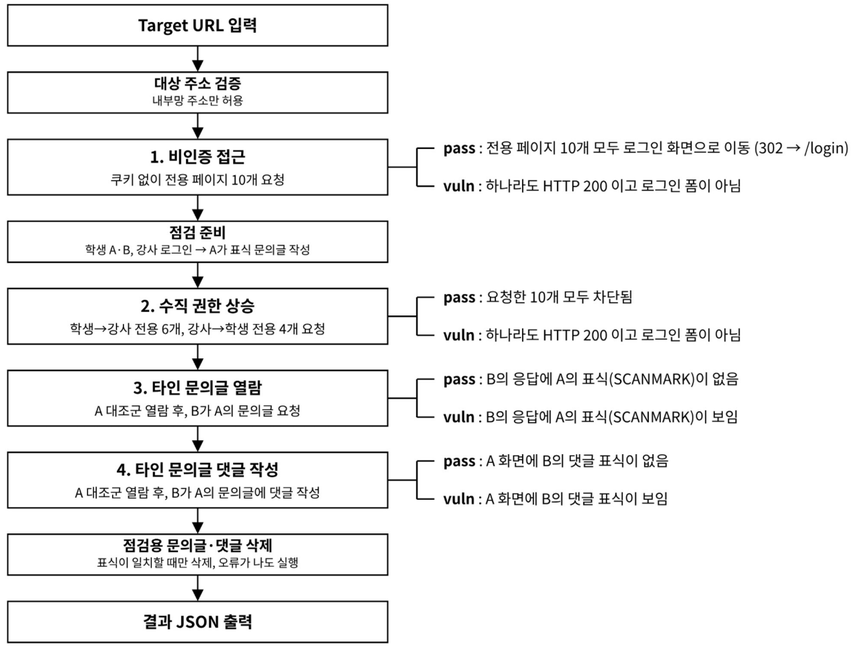
수동 진단 절차

- 계정 A로 로그인해 문의글 작성, 주소의 문의글 ID 확인
- 계정 B로 로그인해 A 문의글 주소를 직접 입력해 열람 확인
- 계정 B로 A 문의글에 댓글 작성, A 화면에서 확인
- 비로그인, 학생 계정으로 전용 페이지 접근 확인

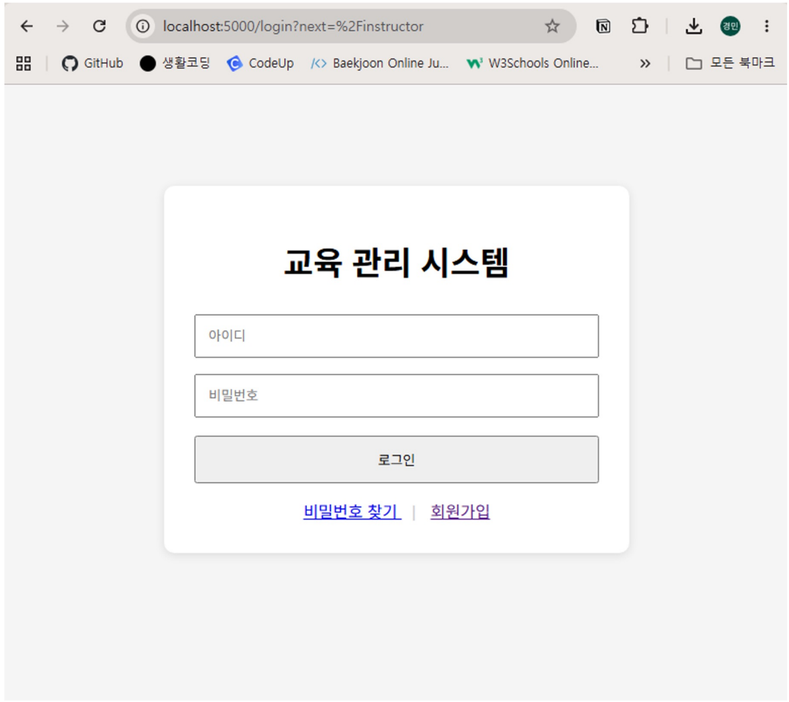
한계점

- 계정을 오가며 반복해야 해서 번거로움
- 화면을 눈으로 보고 판단하므로 진단자별 해석이 다를 수 있음
- 점검용 글이 서비스에 남음

자동 진단 파이프라인



04 취약점 분석 및 시나리오 - 불충분한 권한 검증




```
127.0.0.1 - - [07/Oct/2026 04:21:57] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [07/Oct/2026 04:22:08] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [07/Oct/2026 04:22:08] "GET / HTTP/1.1" 302 -
127.0.0.1 - - [07/Oct/2026 04:22:08] "GET /instructor HTTP/1.1" 200 -
127.0.0.1 - - [07/Oct/2026 04:22:13] "GET /logout HTTP/1.1" 302 -
127.0.0.1 - - [07/Oct/2026 04:22:13] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [07/Oct/2026 04:22:25] "GET /instructor HTTP/1.1" 302 -
127.0.0.1 - - [07/Oct/2026 04:22:25] "GET /login?next=/instructor HTTP/1.1" 200 -
```

점검 항목 1,2 : 전용 페이지 접근 (비인증, 수직권한 상승)

- 점검 1(비인증 접근) : 쿠키 없이 학생, 강사 전용 페이지 10개를 요청
- 점검 2(수직권한 상승) : 학생 세션으로 강사 전용 6개, 강사 세션으로 학생 전용 4개를 요청
- **판정** : 하나라도 HTTP 200이고 로그인 폼이 아니면 vuln, 전부 로그인 화면으로 이동하거나 차단되면 pass

04 취약점 분석 및 시나리오 - 불충분한 권한 검증

```python
def check_unauthenticated_access(url):
    """
    로그인하지 않은 상태에서 학생/강사 전용 페이지 접근이 가능한지 확인
    """
    session = requests.Session()
    content = []
    exposed = []
                (variable) student_paths: list[str]
    for path in student_paths + instructor_paths:
        r = request(session, "GET", url, path)
        content.append(f"GET {path}: {describe(r)}")
        if is_exposed(r):
            exposed.append(path)

    if exposed:
        content.append(f"로그인 없이 노출된 경로 {len(exposed)}개: {', '.join(exposed)}")

    return make_finding("비인증 접근", "/student, /instructor", content, "vulnerable" if exposed else "pass")
```

```python
def is_exposed(r):
    """
    200 응답이면서 로그인 폼이 아니면 보호 자원이 노출된 것으로 판단
    """
    return r.status_code == 200 and not has_login_form(r)
```

점검 1,2 : 주요 함수

- **check_unauthenticated_access()**
  - 쿠키 없이 전용 페이지를 10개 요청하고 노출된 경로를 기록
- **is_exposed()**
  - HTTP 200이고 로그인 폼이 없으면 노출
  - 리다이렉트는 따라가지 않고 HTTP 302 -> /login 형태로 기록

04 취약점 분석 및 시나리오 - 불충분한 권한 검증

```python
def check_vertical_escalation(url, session_student, session_instructor=None):
    """
    학생 계정으로 강사 전용 페이지, 강사 계정으로 학생 전용 페이지 접근이 가능한지 확인
    """
    content = []
    exposed = []

    for path in instructor_paths:
        r = request(session_student, "GET", url, path)
        content.append(f"학생 세션 GET {path}: {describe(r)}")
        if is_exposed(r):
            exposed.append(f"학생→{path}")

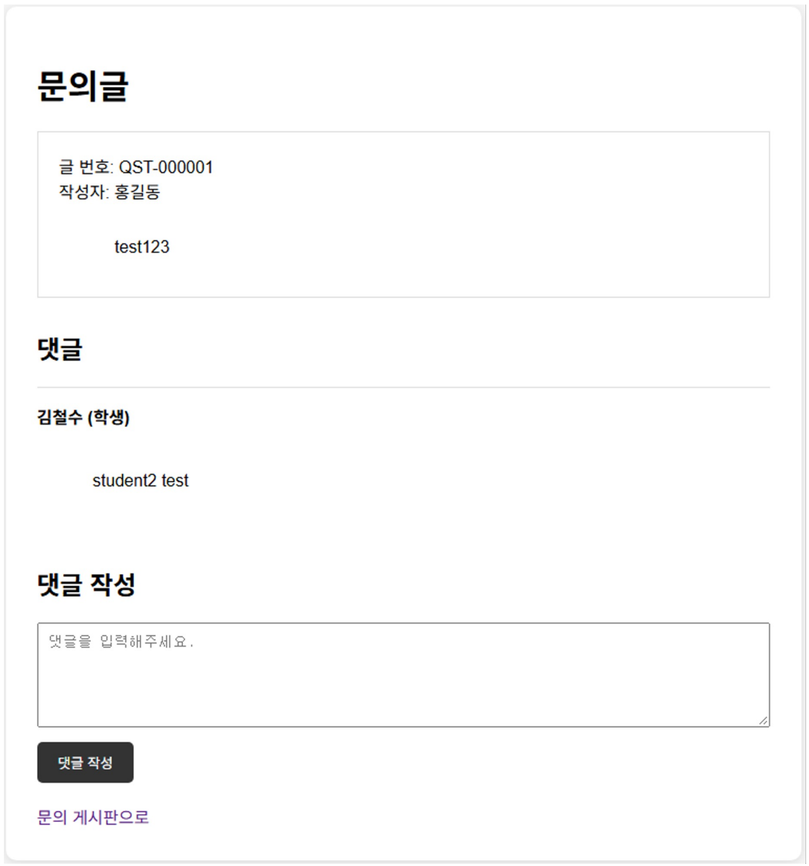
    if session_instructor is None:
        content.append("강사 계정이 없어 강사→학생 방향은 점검하지 않음")
    else:
        for path in student_paths:
            r = request(session_instructor, "GET", url, path)
            content.append(f"강사 세션 GET {path}: {describe(r)}")
            if is_exposed(r):
                exposed.append(f"강사→{path}")

    if exposed:
        content.append(f"권한 밖에서 노출된 전용 경로 {len(exposed)}개: {', '.join(exposed)}")

    return make_finding("수직 권한 상승", "/instructor, /student", content, "vulnerable" if exposed else "pass")
```

- **check_vertical_escalation():**
  - 학생 -> 강사, 강사 -> 학생 방향으로 접근 시도

04 취약점 분석 및 시나리오 - 불충분한 권한 검증



점검 항목 3,4 : 타인 문의글 접근 (열람, 댓글 작성)

- 점검 준비 : 학생 A,B 강사 계정으로 로그인, A가 표식(SCANMARK-...)이 든 문의글을 작성
- 점검 3(열람) : A 본인이 열람 가능한지 대조군으로 먼저 확인한 뒤, B가 A의 문의글을 요청
  - vuln : B의 응답에 A의 표식이 보임
  - pass : 표식이 없음
- 점검 4(댓글 작성) : B가 A의 문의글에 댓글(SCANCOMMENT-...)을 작성
  - vuln : A화면에 B의 댓글 표식이 보임
  - pass : 표식이 없음

04 취약점 분석 및 시나리오 - 불충분한 권한 검증

```python
def check_horizontal_read(url, session_a, session_b, inquiry_id, marker):
    """
    학생 B가 학생 A의 문의글을 열람할 수 있는지 확인
    """
    path = f"/inquiries/{inquiry_id}"

    own = request(session_a, "GET", url, path)
    if not (is_exposed(own) and marker in own.text): # 작성자 본인도 못 읽으면 비교 기준이 성립하지 않음
        return make_finding("타인 문의글 열람", path, ["작성자 본인도 문의글을 열람하지 못해 비교할 수 없음"], "unknown")

    r = request(session_b, "GET", url, path)
    can_read = is_exposed(r) and marker in r.text

    content = [
        f"작성자(A) GET {path}: {describe(own)}",
        f"다른 학생(B) GET {path}: {describe(r)}",
        f"B의 응답에 A의 문의글 내용 포함: {'예' if can_read else '아니오'}"
    ]
    if re.fullmatch(r"[A-Z]{3}-\d{6}", inquiry_id): # 식별자 형식 확인
        content.append(f"문의글 식별자 {inquiry_id}는 일련번호 형식이라 다른 글의 ID도 추측 가능")

    return make_finding("타인 문의글 열람", path, content, "vulnerable" if can_read else "pass")
```

```python
def check_horizontal_write(url, session_a, session_b, inquiry_id, marker):
    """
    학생 B가 학생 A의 문의글에 댓글을 작성할 수 있는지 확인
    """
    path = f"/inquiries/{inquiry_id}/comment"

    own = request(session_a, "GET", url, f"/inquiries/{inquiry_id}")
    if not (is_exposed(own) and marker in own.text):
        return make_finding("타인 문의글 댓글 작성", path, ["작성자 본인도 문의글을 열람하지 못해 비교할 수 없음"], "unknown")

    comment_marker = "SCANCOMMENT-" + secrets.token_hex(4)
    r = request(session_b, "POST", url, path, data={"content": comment_marker})

    after = request(session_a, "GET", url, f"/inquiries/{inquiry_id}") # A 화면에서 B의 댓글 확인
    can_comment = comment_marker in after.text

    content = [
        f"다른 학생(B) POST {path}: {describe(r)}",
        f"작성자(A) 화면에 B의 댓글 표시: {'예' if can_comment else '아니오'}"
    ]

    return make_finding("타인 문의글 댓글 작성", path, content, "vulnerable" if can_comment else "pass")
```

점검 3,4 : 주요 함수

- **prepare_test_data()**
  - 기존계정 로그인, 표식이 든 점검용 문의글 작성
- **check_horizontal_read()**
  - A 대조군 열람 -> B 열람 시도 -> 표식 확인
- **check_horizontal_write()**
  - A 대조군 열람 -> B 댓글 작성 -> A 화면 재조회
- **cleanup_test_data()**
  - 점검 후 표식이 일치하는 점검용 글과 댓글만 삭제

04 취약점 분석 및 시나리오 - 불충분한 권한 검증


```
        "category": "비인증 접근",
        "result": "pass",
        "severity": "low",
        "reason": "비인증 상태에서 /student 및 /instructor 하위 보호 경로에 접근 시 모두 HTTP 302로
 /login 페이지로 리다이렉트되었습니다. 보호 자원 내용이 노출되거나 데이터 변경이 가능하다는 근거가
없으므로 비인증 접근 통제는 양호한 것으로 판단됩니다."
      },
      "needs_review": false
    },
```

```
        ],
        "category": "수직 권한 상승",
        "result": "pass",
        "severity": "low",
        "reason": "학생 권한으로 강사 전용 경로에 접근하거나, 강사 권한으로 학생 전용 경로에 접근하
려는 시도가 모두 리다이렉트 처리되어 보호 자원 접근이 허용된 정황이 확인되지 않았습니다."
      },
      "needs_review": false
    },
```

```
        "category": "타인 문의글 열람",
        "result": "vulnerable",
        "severity": "high",
        "reason": "다른 학생(B)이 작성자(A)의 문의글 상세 경로에 접근했을 때 HTTP 200으로 응답했고,
응답에 A의 문의글 내용이 포함되어 있어 권한 없는 사용자의 타인 문의글 열람이 가능함이 확인됨. 또한
문의글 식별자가 일련번호 형식이라 다른 문의글 ID 추측을 통한 추가 열람 위험도 존재한다."
      },
      "needs_review": false
```

```
        "category": "타인 문의글 댓글 작성",
        "result": "vulnerable",
        "severity": "high",
        "reason": "다른 학생(B)이 타인 문의글(QST-000002)에 댓글 작성 요청을 보낸 뒤 리다이렉트되었
고, 작성자(A) 화면에 B의 댓글이 실제로 표시되어 데이터 변경이 성공한 것으로 확인됩니다. 권한 없는
사용자가 타인의 문의글에 댓글을 작성할 수 있으므로 접근 통제 취약점입니다."
      },
      "needs_review": false
    }
  ],
```


점검 결과

- 점검 1. 비인증 접근 : pass (모든 전용 페이지가 /login으로 이동
- 점검 2. 수직권한 상승 : pass(학생 - 강사 모두 /로 이동
- 점검 3. 타인 문의글 열람 : vulnerable(B가 A의 문의글 열람 가능)
- 점검 4. 타인 문의글 댓글 작성 : vulnerable(B의 댓글이 A의 글에 표시)

04 취약점 분석 및 시나리오 - 정보 누출



가이드 항목

주요정보통신기반시설 취약점 - 정보 누출(위험도 : 상)

판단 기준

- **취약**

  중요 정보가 평문으로 노출되거나, 서비스 운영 및 서버 사이드의 구조를 파악할 수 있는 과도한 정보가 노출되는 경우

- **양호**

  중요 정보가 마스킹 처리되어 있으며, 서비스 운영 및 서버 사이드의 구조를 파악할 수 있는 불필요한 정보가 노출되지 않는 경우

※ **중요 정보** : 고유식별번호(주민등록번호), 비밀번호, 신용정보 등

04 취약점 분석 및 시나리오 - 정보 누출(cont'd)

수동진단 절차 및 한계점

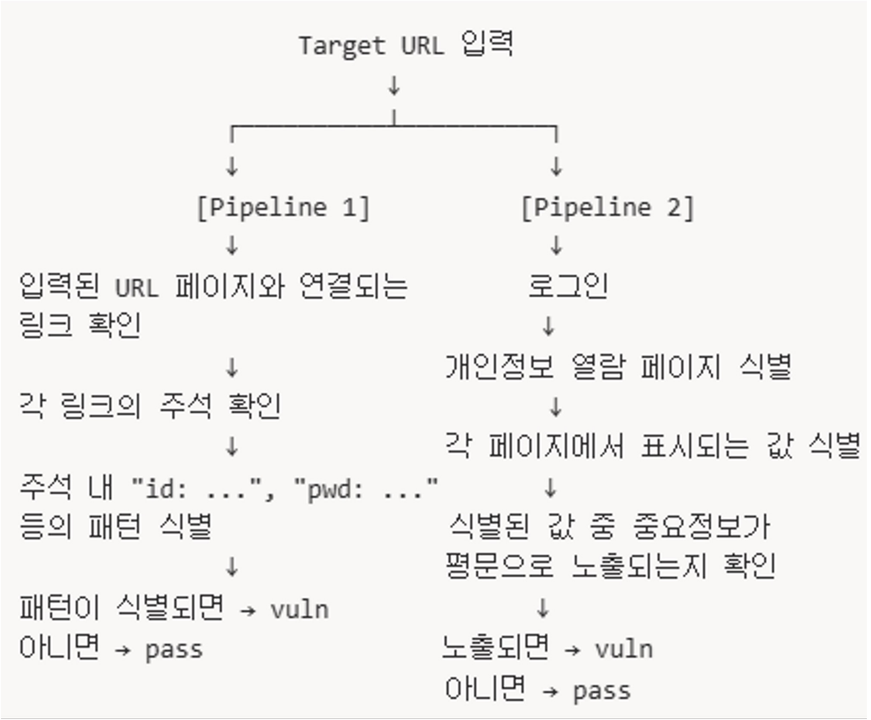
수동 진단 절차

- 확인 가능한 모든 페이지의 html 소스코드 확인
- 로그인 후 개인정보 확인 페이지 접근
- 각 페이지에서 진단 기준에 따른 중요 정보 누출 확인

한계점

- html 소스코드 확인의 번거로움
- 진단 기준에 대한 해석이 진단자 별로 상이할 수 있음

자동 진단 파이프라인



04 취약점 분석 및 시나리오 - 정보 누출(cont'd)

```
<!DOCTYPE html>
<html lang="ko"> view-source
 ▶ <head> ... </head>
 ▼ <body>
   ▼ <div class="container">
       <h1>교육 관리 시스템</h1>
     ▼ <form method="POST">
         <input type="text" name="username" placeholder="아이디" required>
         <!--default id : student1/student123--> == $0
         <input type="password" name="password" placeholder="비밀번호" required>
         <button type="submit"> 로그인 </button>
       </form>
     ▶ <div style="text-align: center; margin-top: 15px;"> ... </div>
     </div>
   </body>
 ▶ <rq-implicit-test-rule-widget class="rq-element" style="display: none;" draggable="t
   test-rule-widget> custom-element
</html>
```

점검 항목 1. 주석

- 주석 내 계정정보나 디버그 정보가 포함되는지 점검
- 로그인 페이지, 로그인 이후 메인 페이지에서 접근 가능한 페이지를 재귀적으로 탐색
- 각 페이지의 주석에서 정규표현식을 통해 중요정보 식별

04 취약점 분석 및 시나리오 - 정보 누출(cont'd)

```
comment_patterns = { # 주석 내 정보확인 패턴
        "account_info": [
            r"\b(?:id|userid|user_id|username|password|passwd|pwd|account)\s*[:=]\s*[^\s<]+",
            r"\b(?:아이디|사용자명|사용자\s*ID|계정|비밀번호|패스워드)\s*[:=]\s*[^\s<]+"
        ],

        "debug_info": [
            r"\b(?:debug|debugging|trace|stack|stack\s*trace|exception|error|log)\b",
            r"\b(?:디버그|디버깅|스택|스택\s*트레이스|예외|에러|로그)\b"
        ]
}
```

주석 진단 주요 함수 및 클래스

- **class WebCrawler**
  - url 입력 후 crawl() 실행 시 url을 기준으로 연결된 링크 식별
  - get_urls()를 통해 입력 url을 포함한 연결된 링크 반환
  - 로그인 전 로그인 페이지, 로그인 후 메인 페이지 대상 crawl
- **check_comment()**
  - WebCrawler로 얻은 각 경로에 접근하여 주석 확인
  - comment_patterns를 기반으로 계정, 디버그 정보 식별

04 취약점 분석 및 시나리오 - 정보 누출(cont'd)



점검 항목 2. 개인정보 열람 페이지

- 마이페이지, 개인정보 수정 등
- 메인 페이지에서 개인정보 열람 페이지로 추정되는 경로 식별
- 각 경로에서 중요정보가 평문으로 노출되는지 확인

04 취약점 분석 및 시나리오 - 정보 누출(cont'd)

```
info_patterns = {
        "account_number": r"\b\d{3,6}[- ]?\d{2,6}[- ]?\d{2,8}\b", # 계좌 번호 패턴,

        "card_number": r"\b(?:\d{4}[- ]?){3}\d{4}\b", # 카드 번호 패턴,

        "resident_registration_number": r"\b\d{6}-\d{7}\b", # 주민등록번호 패턴

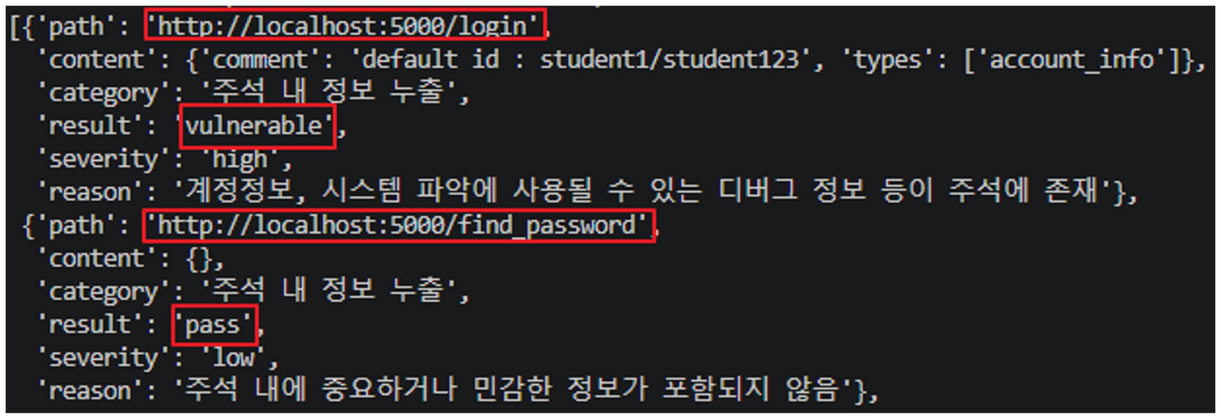
        "pwd_placeholder_pattern" : # 비밀번호 란에 표시되는 것을 허용할 패턴
            r"^(?:입력|입력하세요|입력해 주세요|변경|변경할 비밀번호|새 비밀번호|새로운 비밀번호|현재 비밀번호|확인|비밀번호 확인)$",

        "pwd_masking_pattern" : r"^[*•●xX#]+$" # 비밀번호 마스킹 패턴
}
```

개인정보 열람 페이지 진단 주요 함수

- **find_mypage()**
  - 로그인된 세션의 메인 페이지에서 개인정보를 확인할 수 있을 것으로 추정되는 경로를 키워드 기반으로 식별
  - 실제 식별 경로 : /student/mypage, /student/profile
- **find_personal_info()**
  - find_mypage() 에서 식별된 경로에서 표시되는 개인정보 확인
  - 테이블 또는 <input> 태그를 통해 표시되는 것을 전제로 하며, {"이름" : "김철수"} 형식의 딕셔너리 반환
- **check_personal_info()**
  - find_personal_info() 에서 확인된 개인정보 중 중요 정보가 노출되는지 확인
  - 정규표현식을 이용하여 주민번호, 카드번호, 비밀번호, 계좌번호가 평문으로 노출되는지 식별

04 취약점 분석 및 시나리오 - 정보 누출(cont'd)

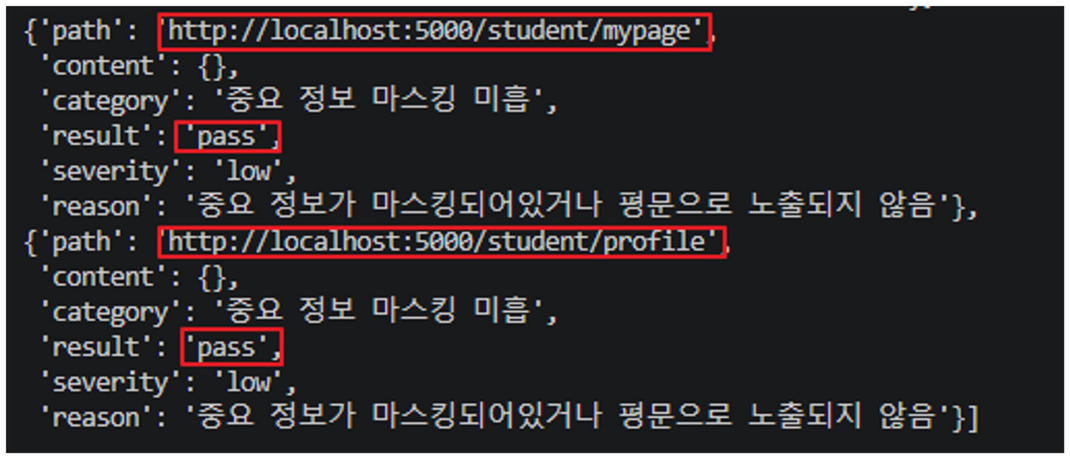


```
[{'path': 'http://localhost:5000/login',
  'content': {'comment': 'default id : student1/student123', 'types': ['account_info']},
  'category': '주석 내 정보 누출',
  'result': 'vulnerable',
  'severity': 'high',
  'reason': '계정정보, 시스템 파악에 사용될 수 있는 디버그 정보 등이 주석에 존재'},
 {'path': 'http://localhost:5000/find_password',
  'content': {},
  'category': '주석 내 정보 누출',
  'result': 'pass',
  'severity': 'low',
  'reason': '주석 내에 중요하거나 민감한 정보가 포함되지 않음'},
```

점검 항목 1. 주석 점검 결과

- 주석에 'default id'가 있는 /login 경로는 'vulnerable'
- 주석에 중요 정보가 없는 경로는 'pass'

04 취약점 분석 및 시나리오 - 정보 누출(cont'd)



```
{'path': 'http://localhost:5000/student/mypage',
 'content': {},
 'category': '중요 정보 마스킹 미흡',
 'result': 'pass',
 'severity': 'low',
 'reason': '중요 정보가 마스킹되어있거나 평문으로 노출되지 않음'},
{'path': 'http://localhost:5000/student/profile',
 'content': {},
 'category': '중요 정보 마스킹 미흡',
 'result': 'pass',
 'severity': 'low',
 'reason': '중요 정보가 마스킹되어있거나 평문으로 노출되지 않음'}]
```

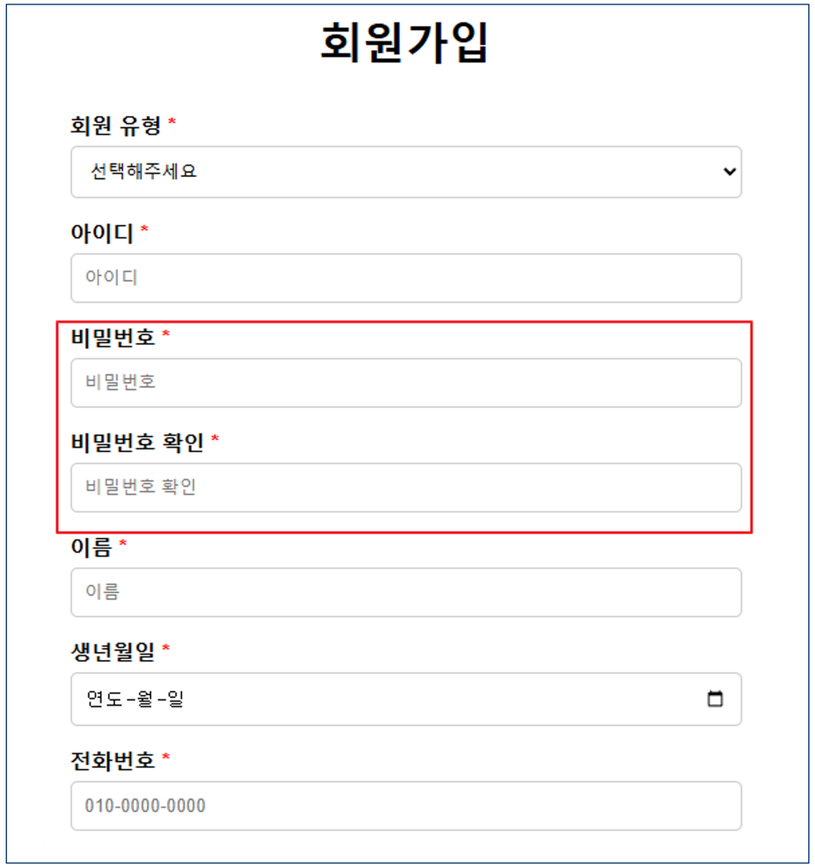
점검 항목 2. 개인정보 열람 페이지 점검 결과

- 개인정보 열람 페이지로 식별한 경로

  : /student/mypage, /student/profile

- 로그인한 상태에서 접근하므로 전화번호 등의 단순 개인정보 노출은 양호한 것으로 판단

04 취약점 분석 및 시나리오 - 약한 비밀번호 정책



가이드 항목

주요정보통신기반시설 취약점 - 약한 비밀번호 정책(위험도 : 상)

판단 기준

- **취약**

  관리자 및 사용자 계정의 비밀번호가 유추하기 쉬운 값으로 설정되어 있거나 낮은 복잡성의 비밀번호 정책이 설정되어 있는 경우

- **양호**

  관리자 및 사용자 계정의 비밀번호가 유추하기 어려운 값으로 설정되어 있거나 높은 복잡성의 비밀번호 정책이 설정되어 있는 경우

04 취약점 분석 및 시나리오 - 약한 비밀번호 정책(cont'd)

수동진단 절차 및 한계점

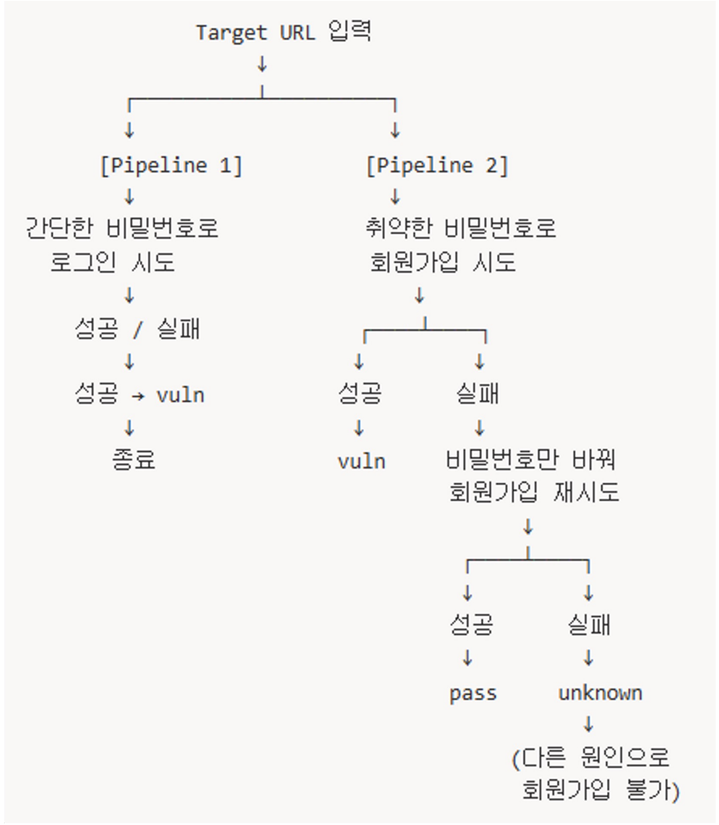
수동 진단 절차

- 특정 id에 대해 간단한 비밀번호로 로그인 시도
- 취약한 비밀번호로 회원가입 시도

한계점

- '유추 가능한 비밀번호'의 모호성
- 로그인 및 회원가입 반복 작업의 번거로움

자동 진단 파이프라인



04 취약점 분석 및 시나리오 - 약한 비밀번호 정책(cont'd)



점검 항목 1. 유추 가능한 비밀번호

- 대상 ID : student1
- 유추 가능한 문자 + 숫자 조합을 생성하여 로그인 시도
- 로그인에 성공하면 취약, 실패 시 양호

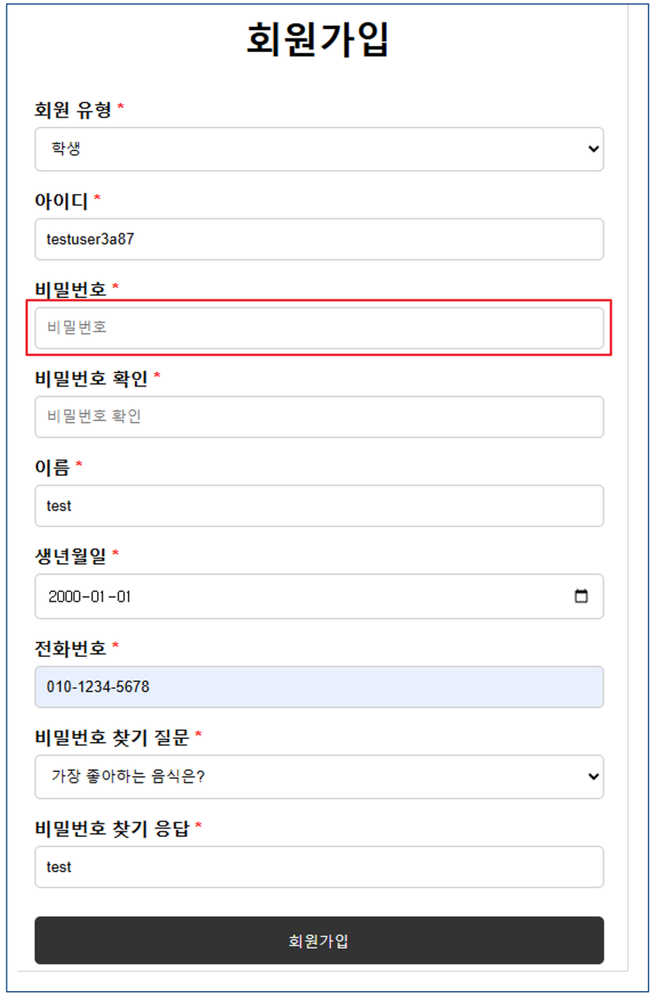
04 취약점 분석 및 시나리오 - 약한 비밀번호 정책(cont'd)

```
vuln_pwd_pattern = {
    "char" : ["student", "qwer", "asdf", "abcd", "admin",
              "administator", "rhksflwk", "instructor",
              "teacher", "mentor", "tutor", ""],
    "num" : ["1", "2", "3", "4", "123", "321", "1111", "2222", ""]
}
```

유추 가능한 비밀번호 진단 주요 함수

- **check_vuln_login()**
  - default id : student1
  - 패턴을 통해 문자 + 숫자 조합을 생성하여 로그인 시도

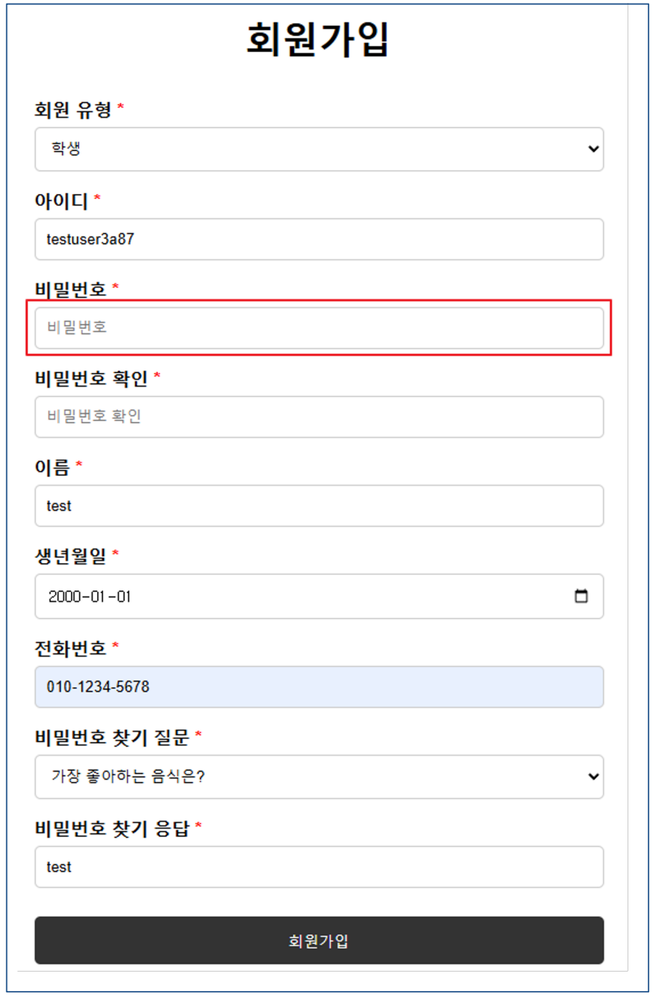
04 취약점 분석 및 시나리오 - 약한 비밀번호 정책(cont'd)



점검 항목 2. 비밀번호 정책 복잡성

- 회원가입 형식 자동 기입
  - username : testuser + (랜덤값)
  - 비밀번호 : 1234
- 회원가입 시도
- 회원가입 성공 시 취약

04 취약점 분석 및 시나리오 - 약한 비밀번호 정책(cont'd)



점검 항목 2. 비밀번호 정책 복잡성

- 회원가입 실패 시 비밀번호 변경
  - s@feP4ssW0rd
  - 이외의 값 유지
- 회원가입 성공 시 pass, 실패 시 unknown

04 취약점 분석 및 시나리오 - 약한 비밀번호 정책(cont'd)

```python
if input_type == "password" or "password" in name:
    return "1234"

if (
    "username" in name
    or "user_id" in name
    or name == "id"
    or "login" in name
):
    return "testuser" + str(uuid.uuid4())[:4] # 반복 테스트 시 충돌을 막기 위해 랜덤값 추가
```

비밀번호 정책 복잡성 진단 주요 함수

- **generate_value()**
  - 회원가입 데이터 생성 함수
  - 각 폼 태그의 name/type 기반으로 테스트용 값 생성
  - username : testuser + (랜덤 값)

    password : 1234
- **fill_signup_form()**
  - 회원가입 페이지 분석 함수
  - form을 분석하고 generate_value()를 호출하여 회원가입 데이터 생성
- **check_vuln_register()**
  - fill_signup_form()으로 생성된 데이터로 회원가입 시도
  - 성공 시 취약으로 판단

    실패 시 비밀번호 변경 (s@feP4ssW0rd) 후 다시 시도
  - 성공 시 양호, 실패 시 unknown

04 취약점 분석 및 시나리오 - 약한 비밀번호 정책(cont'd)


```
[{'path': 'http://localhost:5000/login',
  'content': ['id:student1 password:student123'],
  'category': '유추 가능한 비밀번호',
  'result': 'vulnerable',
  'severity': 'high',
  'reason': '관리자 및 사용자 계정의 비밀번호가 유추하기 쉬운 값으로 설정되어 있음'},
 {'path': 'http://localhost:5000/register',
  'content': [[{'username': 'testuseref9c', 'password': '1234'}]],
  'category': '낮은 복잡도의 비밀번호 정책',
  'result': 'vulnerable',
  'severity': 'high',
  'reason': '1234와 같은 간단한 비밀번호로 회원가입 가능'}]
```


점검 결과

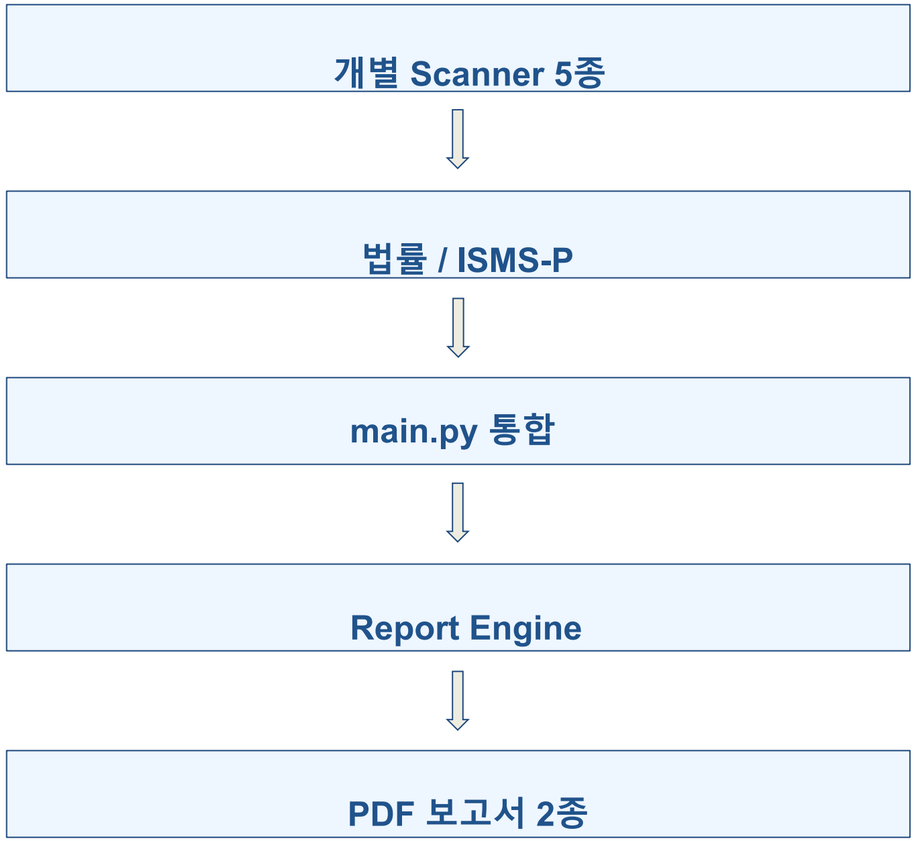
- /login 에서 student1/student123 으로 로그인 성공 -> 취약
- /register 에서 testueref9c/1234 로 로그인 성공 -> 취약

04 취약점 분석 및 시나리오 - 통합 자동화 개요

구현 배경

- 프로젝트에서 5종의 취약점 진단 모듈을 각각 구현
- 각 모듈의 결과를 하나로 통합할 실행 구조 필요
- 진단 결과가 법률, ISMS-P 분석과 보고서 생성까지 이어져야 함

main.py 1회 실행으로
전체 과정 자동 수행



04 취약점 분석 및 시나리오 - 통합 실행 구조

통합 대상 Scanner

- 정보 누출
- 약한 비밀번호 정책
- 불충분한 인증 절차
- 불충분한 권한 검증
- 취약한 비밀번호 복구 절차

실제 통합 실행 화면

```
(venv) C:\Users\ez\34_module_proj2>python vuln_assessment\scanner\main.py
[1/5] 정보 누출 진단 시작
[2/5] 약한 비밀번호 진단 시작
[3/5] 불충분한 인증 절차 진단 시작
[4/5] 불충분한 권한 검증 진단 시작
[5/5] 비밀번호 복구 절차 진단 시작

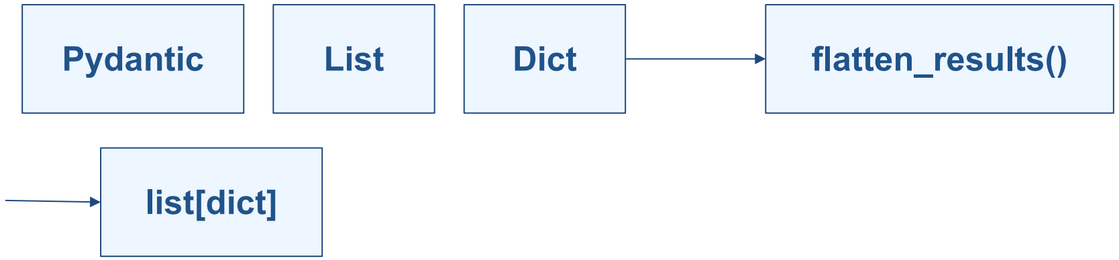
===== 통합 진단 결과 =====
{
  "target": "http://localhost:5000/",
  "results": [
```

→ 하나의 명령으로 다섯개의 Scanner가 순차적으로 진행됨을 확인

04 취약점 분석 및 시나리오 - 결과 통합 및 오류처리

결과 형식 표준화

Scanner마다 서로 다른 반환 형식을 공통 구조로 변환하여
scanner_result.json으로 통합 저장



오류 격리

- 각 Scanner를 독립적인 try-except 구조로 실행
- 특정 모듈 오류가 전체 진행을 중단하지 않도록 구성
- 발생 오류는 errors 항목에 별도 기록


```
{
  "path": "http://localhost:5000/login",
  "content": [
    "id:student1 password:student123"
  ],
  "category": "유추 가능한 비밀번호",
  "result": "vulnerable",
  "severity": "high",
  "reason": "관리자 및 사용자 계정의 비밀번호가 유추하기 쉬운 값으로 설정되어 있음"
},
{
  "path": "http://localhost:5000/register",
  "content": [
    "id:testuser password:1234"
  ],
  "category": "낮은 복잡도의 비밀번호 정책",
  "result": "vulnerable",
  "severity": "high",
  "reason": "1234와 같은 간단한 비밀번호로 회원가입 가능"
},
```


- 통합 진단 결과 예시


```
      "severity": "high",
      "reason": "임시 비밀번호가 화면에 평문으로 노출되었고, 고정된 패턴
    }
  ],
  "errors": []
}
```


- Scanner 오류 없이 실행 완료 화면

04 취약점 분석 및 시나리오 - 후속 모듈 연계

모듈간 데이터 자동 전달

scanner_result.json

↓

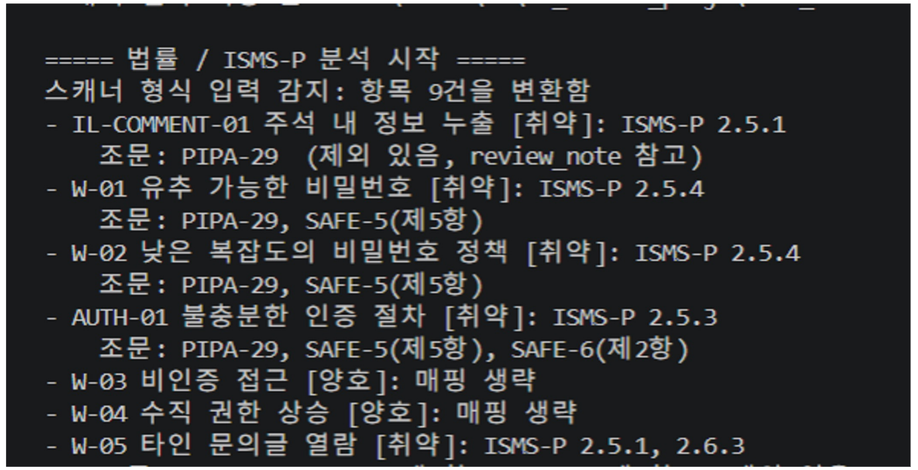
Legal Module

↓

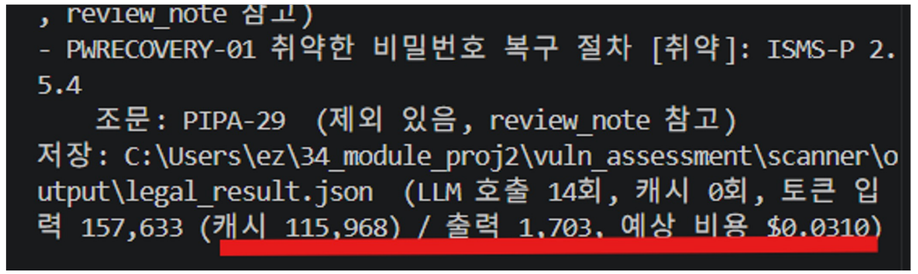
ISMS-P / 관련 법령 매핑

↓

legal_result.json



- 법률 / ISMS-P 분석 결과



- legal_result.json 저장 완료 화면

04 취약점 분석 및 시나리오 - 통합 테스트 결과

통합 테스트 결과

scanner	전체 결과	오류
5종	9건	0건

취약	양호	PDF
7건	2건	2종

최종 실행 흐름

main.py -> 5종 진단 -> 법률 ISMS-P 분석 -> Report Engine

-> PDF 보고서 2종

최종 보고서 생성

```
===== 보고서 생성 시작 =====
[카탈로그] IL-COMMENT-01 '주석 내 정보 누출' → 카탈로그 IL 항목 사용 (ID 앞부분 일치)
[카탈로그] W-01 '유추 가능한 비밀번호' → 카탈로그에 없음 (실무진용 판단 기준·조치 방법 칸이 비어 나
[카탈로그] W-02 '낮은 복잡도의 비밀번호 정책' → 카탈로그에 없음 (실무진용 판단 기준·조치 방법 칸이
[카탈로그] AUTH-01 '불충분한 인증 절차' → 카탈로그 IA 항목 사용 (이름 일치)
[카탈로그] W-03 '비인증 접근' → 카탈로그에 없음 (실무진용 판단 기준·조치 방법 칸이 비어 나옴)
[카탈로그] W-04 '수직 권한 상승' → 카탈로그에 없음 (실무진용 판단 기준·조치 방법 칸이 비어 나옴)
[카탈로그] W-05 '타인 문의글 열람' → 카탈로그에 없음 (실무진용 판단 기준·조치 방법 칸이 비어 나옴)
[카탈로그] W-06 '타인 문의글 댓글 작성' → 카탈로그에 없음 (실무진용 판단 기준·조치 방법 칸이 비어 L
[카탈로그] PWRECOVERY-01 '취약한 비밀번호 복구 절차' → 카탈로그 PR 항목 사용 (이름 일치)
[facts] 취약 7건 / 전체 9개, 위험 수준 심각
[exposure] 법정 상한 150,000,000원, 후보 사례 19건 → C:\Users\ez\34_module_proj2\vuln_assessment\re
[llm] 호출 1회...
[render] C:\Users\ez\34_module_proj2\vuln_assessment\report_engine\out\SCAN-20261006-001\exec_repor
[render] C:\Users\ez\34_module_proj2\vuln_assessment\report_engine\out\SCAN-20261006-001\tech_repor

보고서 생성 완료
```

생성된 PDF 보고서는 자동으로 열리도록 구성하여
최종 결과까지 즉시 확인 가능

04 ISMS-P 연계 및 대응 - 법령 매핑 흐름

스캐너가 찾은 취약점마다 어떤 ISMS-P 인증기준을 못 지켰고, 개인정보 보호법·안전성 확보조치 기준의 어느 조문·항에 어긋나는지 자동으로 붙여 보고서 모듈에 넘긴다

STEP 01

스캐너 결과 입력

스캐너 공통 JSON 그대로 입력

양호 항목은 조문 없이 통과

STEP 02

기준 선택 (LLM)

ISMS-P 101개 기준 목록 안에서

해당 기준 선택 + 이유 작성

목록 밖 번호는 코드가 제거

STEP 03

조문 매핑 (규칙)

선택된 기준의 [관련 법규] 조문을

법령 txt에서 원문째 복사

AI가 조문을 추가할 수 없음

STEP 04

항 선택·검증

LLM이 진단 근거에 맞는 항만 선택

원문에 없는 항·무관한 조문 제거

제외 사유는 review_note에 기록

조문 선택권은 AI에게 없다. 조문은 관련 법규 표에서만 오고, AI는 그 안에서 기준과 항을 고르며, 코드가 목록 밖 값을 버린다

근거 자료: 개인정보 보호법(법률 제21445호) · 개인정보의 안전성 확보조치 기준(고시 제2026-9호) · ISMS-P 인증기준 안내서(2023.11, 101개 기준)

04 ISMS-P 연계 및 대응 - ① 항 단위 판단

```python
# LLM 2차 응답 형식 (pydantic) - 자유 텍스트 파싱 없음
class ParagraphPick(BaseModel):
    law_code: str              # 제시된 조문 코드 그대로 (SAFE-6)
    applicable: bool           # 진단 근거가 이 조문 의무에 어긋나면 true
    paragraphs: list[str]      # 해당 항 ["제3항"]
    reason: str

resp = client.chat.completions.parse(
    model="gpt-5.5", messages=messages,
    response_format=ParagraphSelection)   # 구조화 출력

# 닫힌 목록 검증: 제시한 조문, 원문에 있는 항만 통과
valid = {c["law_code"]: {label for label, _ in c["paragraphs"]}
         for c in candidates}
for item in resp.laws:
    if item.law_code not in valid:        # 제시 안 한 조문은 무시
        continue
    labels = [norm_para_label(p) for p in item.paragraphs]
    #   "3", "③", "3항" → "제3항"
    labels = [l for l in labels if l in valid[item.law_code]]
    #   원문에 없는 항은 버림
```

왜 조문이 아니라 항인가

고시 제6조(접근통제)는 ①~⑤항인데 IDOR에 맞는 건 ③뿐.

조문 전체를 붙이면 보고서가 부풀려진다

어떻게 신뢰하나

- **구조화 출력**

  LLM 응답을 pydantic 모델로 직접 받음 (필드 고정)

- **닫힌 목록 3중 검증**

  101개 기준 목록 · 관련 법규 조문 · 원문 항 번호 밖은 버림

- **조문 원문은 복사만**

  text는 법령 txt 그대로. AI 문장은 reason·review_note만

Python · OpenAI gpt-5.5 structured output · pydantic v2 · pdfplumber(법제처·KISA PDF → txt) · 단위 테스트 38건

04 ISMS-P 연계 및 대응 - ② 일관성·검증 장치

판단 기준 공통화	팀 합의 판단 기준(criteria.md)을 1차·2차 프롬프트에 주입. 실행마다 판정이 흔들리던 문제를 기준으로 고정
원문 복사·현행본	법제처 PDF를 txt로 변환해 조문 원문을 복사. 부칙 제외, 같은 조문이 두 벌이면 현행본, 2027년 시행 예정은 in_force false
불일치·제외 기록	같은 유형 항목끼리 결과를 비교해 다르면 '판단 불일치' 표시. AI가 뺀 조문·API 실패는 review_note, 최종 확인은 reviewed
상위 의무 유지	고시 조문이 하나라도 남으면 개인정보 보호법 제29조(안전조치의무)는 코드가 유지. AI가 빼도 복원
캐시·비용	입력 해시로 응답 캐시(같은 입력은 API 없이 즉시). 101개 기준 목록을 프롬프트 앞에 둬 프롬프트 캐싱. 취약 1건당 호출 2회·약 10초

04 ISMS-P 연계 및 대응 - 매핑 결과


```
{
  "item_id": "IL-COMMENT-01",
  "title": "주석 내 정보 누출",
  "status": "취약",
  "evidence": "http://localhost:5000/login 주석에 student1/student123 노출 ...",
  "isms_p": [
    { "criterion_id": "2.10.3", "title": "공개서버 보안",  "mapped_by": "llm" },
    { "criterion_id": "2.5.4",  "title": "비밀번호 관리",  "mapped_by": "llm" }
  ],
  "violated_laws": [
    { "law_code": "PIPA-29", "law_name": "개인정보 보호법",
      "article": "제29조", "article_title": "안전조치의무", "paragraph": null,
      "in_force": true, "mapped_by": "rule+llm",
      "reason": "주석에 계정 정보가 노출되어 안전성 확보에 필요한 기술적·관리적
                 조치 의무에 어긋남 (근거: ISMS-P 2.5.4 관련 법규)" },
    { "law_code": "SAFE-5", "law_name": "개인정보의 안전성 확보조치 기준",
      "article": "제5조", "article_title": "접근 권한의 관리", "paragraph": "제5항",
      "in_force": true, "mapped_by": "rule+llm",
      "reason": "인증수단을 안전하게 적용·관리해야 하는 의무에 어긋남.
                 제1~4항·제6항은 진단 근거에서 확인되지 않음" }
  ],
  "review_note": null
}
```


입력: 정보 누출 스캐너 결과

로그인 페이지 HTML 주석에 테스트 계정 노출 → 취약(상)

매핑 결과

- **ISMS-P 인증기준**

  2.10.3 공개서버 보안 · 2.5.4 비밀번호 관리

- **위반 조문·항**

  개인정보 보호법 제29조(안전조치의무)

  안전성 확보조치 기준 제5조 제5항(인증수단 안전 관리)

- **같은 스캔의 마이페이지 본인 정보 표시 건**

  조문 없음, 사유만 기록 (판단 기준 B1: 본인에게 본인 정보 표시는 유출이 아님)

04 보고서 생성 - 설계 원칙

① 왜 엔진이 필요한가 LLM 단독 → 보고서 엔진

× 과징금 금액을 잘못 옮김

↓

✓ **금액은 LLM에 안 넘김**

× 없는 조문·사례 인용

↓

✓ **ID로만 참조 → 코드 검증**

× 무관한 사례 연결

↓

✓ **후보 안에서만 사례 선택**

× 양호 항목을 취약으로 보고

↓

✓ **취약·양호 판정은 코드 값**

② 판정은 코드로 LLM이 판단하면 안 되는 3가지

사례 지식베이스

- 개인정보위 처분 보도자료 15건 → 사례 카드 30장
- 4단계 등급 검색: 동일 결함 / 같은 유형 / 관련 통제 / 참고
- '참고' 등급은 인용 금지 — 무관한 사례가 근거처럼 보이지 않게

과징금 노출

- 2026.9 시행 개정 개인정보 보호법 기준
- 법정 상한 매출 3% · 투자 감경 최대 40% · ISMS-P 감경 최대 30%
- 유사 처분액 분포 병기 + '상한은 예상 부과액 아님' 경고

조치 우선순위

- 심각도(상3·중2·하1) × 개인정보 자산(1.0 / 0.3)
- × (1 + 0.5 × 사례 근거 점수) — 최대 1.5배
- 공식은 실무진용 보고서 부록 4.4에 그대로 공개

③ 프롬프트 설계 5원칙

역할 분리

판정은 코드가 끝내고 LLM은 문장만 작성 (시스템 규칙 16개)

대조 예시

나쁨/좋음 예시 문장으로 기대 수준 제시 (Few-shot)

JSON 출력

출력 스키마 고정 + JSON 모드, temperature 0.2

후보 주입

검색된 사례 후보만 넣고 그 안에서만 인용 (RAG)

금액 제외

원화 금액은 아예 넘기지 않고 ID로만 참조하게 함

✓ **판정은 코드가, 문장은 AI가, 숫자·조문·조치 방법은 원본 데이터에서 그대로 가져옵니다.**

04 보고서 생성 - 기술 스택과 데이터

④ 기술 스택 문장 생성 · 검색 · 렌더링 · 계산

구분	사용 기술	설정 · 특징	역할	코드 위치
문장 생성 LLM	OpenAI GPT-4o	temperature 0.2, JSON 모드, 진단당 1회	보고서 문장 작성	*engine.py call_llm()*
임베딩 · 검색	text-embedding-3-small + ChromaDB	코사인 유사도 + 메타데이터 필터	유사 처분 사례 검색	*case_kb.py*
템플릿	Python 3.12 + Jinja2	경영진용 · 실무진용 템플릿 2개	같은 데이터로 HTML 조립	*templates/*.j2*
PDF 렌더링	WeasyPrint (+ Pango)	HTML · CSS → PDF, Pretendard 임베드	PDF 2종 출력	*engine.py render()*
데이터 변환	openpyxl	드롭다운 값 · 필수값 · 중복 검사	사례 시트 → JSON	*convert_cases.py*
금액 계산	Python (규칙 기반)	개정 개인정보 보호법 기준 상수	과징금 노출 · 우선순위	*exposure.py*

LLM은 설정만 바꾸면 Claude로 교체 가능 — 호출 코드는 call_llm() 하나뿐

⑤ 데이터 구축 보고서가 근거로 쓰는 원본

1. 사례 카드화

- 개인정보위 처분 보도자료 PDF 15건 (2023~2026) 수집
- 엑셀 시트에 처분 사례 카드 30장으로 정리
- 기관·처분일·사실관계·처분 결과·과징금·출처
- 유형·공격 태그·ISMS-P 기준은 드롭다운으로만 선택

2. 검사 · 변환

- convert_cases.py가 시트를 검사 후 cases.json 생성
- 필수값 · 날짜 형식 · 사례 ID 형식과 중복 확인
- 금액 칸 · 원본 PDF 존재 여부까지 점검
- 오류가 하나라도 있으면 JSON을 만들지 않음

3. 벡터 DB 적재

- 사례가 짧아 청크 분할 없이 카드 1장 = 문서 1개
- 취약점 유형·ISMS-P 기준은 메타데이터 플래그로 저장
- 검색 시 등급별 필터로 사용
- 적재는 매번 새로 — 지운 내용이 남지 않게

4. 항목 카탈로그

- KISA 2026 상세가이드 웹 21개 항목 원문
- 판단 기준 · 조치 방법을 JSON 카탈로그로 정리
- 실무진용 보고서에 원문 그대로 사용
- 요약·재작성 없이 인용해 출처를 보존

04 보고서 생성 - 생성 흐름과 검증

⑥ 보고서 생성 흐름 코드 → LLM 1회 → 검증 → PDF 2종

STEP 01

코드가 판정

ISMS-P 매핑 모듈 출력 JSON을 받아 취약 항목 통계 · 전체 위험 수준 · 유사 사례 검색 · 과징금 노출액을 전부 코드로 계산합니다. 개인정보 처리 자산에 '상' 취약점이 있으면 위험 수준은 '심각'.

STEP 02

LLM 1회 호출

판정이 끝난 사실만 넘겨 경영진용 한 줄 결론 · 핵심 위험 · 조치 로드맵 같은 문장만 쓰게 합니다. 금액은 넘기지 않습니다.

STEP 03

검증

LLM이 쓴 취약점 · 법령 · 사례 ID를 실제 데이터와 대조해 없는 ID는 지우고, 로드맵에서 빠진 취약점은 코드가 채웁니다.

STEP 04

PDF 2종 생성

같은 데이터를 템플릿 두 개에 넣어 경영진용 슬라이드와 실무진용 상세 보고서를 만듭니다.

⑦ 검증 코드 engine.py validate() LLM 출력은 코드 대조를 통과한 뒤에만 보고서로

engine.py · validate() — LLM 출력 검증, 실제 코드 발췌

```
① 실제 데이터에 있는 ID만 남기고 나머지는 경고와 함께 제거
def keep(ids, valid, what, where):
    good = [i for i in ids or [] if i in valid]
    for bad in set(ids or []) - set(good):
        warnings.append(f"{where}: 존재하지 않는 {what} '{bad}' 제거")
    return good

② 법령은 실제 매핑만, 사례는 그 위험의 검색 후보(참고 등급 제외) 안에서만
    r["law_codes"] = keep(r.get("law_codes"), facts["laws"], "law_code", where)
    allowed = {h["case_id"]: h["match_level"] for fid in r["finding_ids"]
               for h in facts["case_candidates"].get(fid, [])}
    allowed = {c: lv for c, lv in allowed.items() if lv in STRONG_LEVELS}
    r["case_ids"] = keep(r.get("case_ids"), allowed, "case_id(이 위험의 인용 가능 후보가 아님)",
where)[:2]

③ 문장에 원화 금액이 섞이면 경고 (금액은 템플릿이 원본에서 채움)
money = re.compile(r"\d[\d,.]*\s*(억|천만|백만|만)?\s*원")
texts = [("headline", llm.get("headline", "")), ("summary", llm.get("summary", ""))]
texts += [(f"top_risks[{i}]", r.get("impact", "") + " " + r.get("title", "")) for i, r
in enumerate(risks)]
texts += [(f"decisions[{i}]", d) for i, d in enumerate(llm.get("decisions", []))]
for where, t in texts:
    if money.search(t or ""):
        warnings.append(f"{where}: 문장에 금액이 들어 있음 → '{money.search(t).group(0)}' (금액
은 템플릿이 채워야 함)")
```

01 **없는 취약점 · 조문 · 사례 ID 제거**
keep()가 원본 facts에 실제로 있는 ID만 남기고 나머지는 경고와 함께 버립니다. 유효한 취약점이 하나도 없는 위험 항목은 통째로 삭제합니다.

02 **후보 밖 · 참고 등급 사례 인용 차단**
그 위험에 묶인 취약점의 검색 후보 중 '참고'를 뺀 등급 안에 있어야만 인정하고, 없으면 최고 등급 후보를 대신 연결합니다.

03 **로드맵 누락 취약점 자동 추가**
LLM이 로드맵에서 빠뜨린 취약점은 코드가 채워 넣습니다.

04 **문장 속 원화 금액 경고**
정규식으로 탐지. 자동 삭제는 하지 않고 경고만 남기므로 사람이 확인해야 합니다.

걸린 항목은 run_log.json에 경고로 기록되어 보고서마다 무엇이 걸렸는지 추적할 수 있습니다.

04 보고서 생성 - 경영진용 보고서

⑧ 경영진용 보고서 SCAN-20261007-002-E · 16:9 8장 중 표지·4쪽(넘침)을 뺀 6장. 문장은 LLM, 위험 수준·건수·금액은 코드 판정

2쪽 · 진단 결론 한 줄 결론은 LLM, 수치는 코드

SK Shieldus Rookies

진단 결론

외부인이 강의 수강 정보를 쉽게 볼 수 있습니다

러닝브릿지의 온라인 교육 플랫폼은 120,000명의 회원 정보를 처리하고 있습니다. 그러나 현재 시스템에서는 외부인이 강의 수강 정보를 포함한 개인정보에 접근할 수 있는 취약점이 발견되었습니다. 이러한 취약점은 개인정보 유출로 이어질 수 있으며, 이는 법적 책임과 신뢰 손실로 이어질 위험이 있습니다.

전체 위험 수준	취약 항목	영향받는 개인정보 처리 자산
심각	7 건 / 점검 19개 항목	1 개
개인정보를 처리하는 자산에 '상' 위험 취약점이 있음	상 7건, 중 0건, 하 0건	웹 서비스 (localhost:5000)

3쪽 · 핵심 위험 위험 3건에 법령·사례 병기

SK Shieldus Rookies

핵심 위험

1 외부인이 강의 수강 정보를 쉽게 볼 수 있음 상

주석 내 정보 누출로 인해 외부인이 시스템 구조를 파악하고, 회원의 개인정보에 접근할 수 있는 위험이 있습니다.

유사 사례 대구한의대학교 · 권한 확인 누락, 개인정보 노출 동일 결함 과태료 360만 원
유사 사례 (주)골프존 · 권한 확인 누락, 개인정보 노출 동일 결함 과징금 75억 400만 원, 과태료 540만 원, 공표명령

2 누구나 추측할 수 있는 비밀번호로 가입할 수 있음 상

유추 가능한 비밀번호와 낮은 복잡도의 비밀번호 정책으로 인해 계정 탈취 및 개인정보 유출 위험이 높습니다.

관련 법령 개인정보 보호법 제29조(안전조치의무) 검토 전
관련 법령 개인정보의 안전성 확보조치 기준 제5조 제5항(접근 권한의 관리) 검토 전
유사 사례 숙명여자대학교 · 비밀번호 규칙 미흡, 본인확인 미흡 동일 결함 과징금 3,750만 원, 과태료 300만 원, 공표
유사 사례 듀오정보 주식회사 · 로그인 실패 제한 없음, 비밀번호 규칙 미흡 동일 결함 과징금 11억 9,700만 원, 과태료 1,320만 원, 공표명령

3 재인증 없이 개인정보 변경 가능 상

불충분한 인증 절차로 인해 외부인이 재인증 없이 회원의 개인정보를 변경할 수 있는 위험이 있습니다.

관련 법령 개인정보 보호법 제29조(안전조치의무) 검토 전
관련 법령 개인정보의 안전성 확보조치 기준 제5조 제5항(접근 권한의 관리) 검토 전
관련 법령 개인정보의 안전성 확보조치 기준 제6조 제2항(접근통제) 검토 전
유사 사례 루이비통코리아(유) · 권한 확인 누락, 본인확인 미흡 동일 결함 과징금 213억 8,500만 원, 공표명령

5쪽 · 법적 근거와 유사 사례 매핑 24건 모두 '검토 전'

SK Shieldus Rookies

법적 근거와 유사 사례

관련 법령과 인증기준

조항·기준	관련 취약점	상태
개인정보 보호법 제29조(안전조치의무)	6건	검토 전
개인정보의 안전성 확보조치 기준 제5조 제5항(접근 권한의 관리)	5건	검토 전
개인정보의 안전성 확보조치 기준 제6조 제2항(접근통제)	4건	검토 전
ISMS-P 2.10.3 공개서버 보안	1건	검토 전
ISMS-P 2.5.4 비밀번호 관리	3건	검토 전
ISMS-P 2.5.3 사용자 인증	2건	검토 전
ISMS-P 2.6.3 응용프로그램 접근	2건	검토 전
ISMS-P 3.2.4 개인정보 목적 외 이용 및 제공	1건	검토 전

유사 처분·보도 사례

과징금 처분 개인정보위 2023-10-12 동일 결함
대구한의대학교 · 권한 확인 누락, 개인정보 노출
과태료 360만 원

과징금 처분 개인정보위 2024-05-09 동일 결함
(주)골프존 · 권한 확인 누락, 개인정보 노출
과징금 75억 400만 원, 과태료 540만 원, 공표명령

과징금 처분 개인정보위 2023-10-12 동일 결함
숙명여자대학교 · 비밀번호 규칙 미흡, 본인확인 미흡
과징금 3,750만 원, 과태료 300만 원, 공표

과징금 처분 개인정보위 2026-04-23 동일 결함
듀오정보 주식회사 · 로그인 실패 제한 없음, 비밀번호 규칙 미흡
과징금 11억 9,700만 원, 과태료 1,320만 원, 공표명령

외 1건은 앞 장의 핵심 위험별 유사 사례에 있습니다.

'검토 전'은 AI가 매핑하고 담당자가 아직 확인하지 않은 항목입니다. 확정 전까지는 위반 소지로 보아야 합니다.
사례 등급: 동일 결함 = 인증기준과 취약점 유형이 모두 같음, 같은 유형 = 취약점 유형이 같음, 관련 통제 = 인증기준만 같음.

6쪽 · 과징금 노출과 감경 여지 상한 1억 5,000만 원

SK Shieldus Rookies

과징금 노출과 감경 여지

최대 노출액 (법정 상한)	투자 감경 반영 시 (예시)	10% 특례 시나리오
1억 5,000만 원	9,000만 원	5억 원
연 매출 50억 원의 3%. 제29조(안전조치의무)에 취약점 6건이 걸려도 상한은 한 번만 적용	보호 투자·조치 기록이 있으면 최대 40% 감경. 지금은 기록이 없어 감경 근거가 없음	반복 위반, 1천만 명 이상 피해, 시정명령 불이행으로 유출될 때만. 현재 요건 해당 없음

같은 유형 결함의 실제 과징금

유사 처분 15건 · 최소 329만 원 · 중앙값 5억 300만 원 · 최대 4,235억 7,500만 원

사례	처분일	과징금
쿠팡 주식회사 (유출 관련) · 권한 확인 누락, 본인확인 미흡 동일 결함	2026-06-11	4,235억 7,500만 원
(주)케이에스한국고용정보 · 파라미터 변조, 관리자 페이지 노출 동일 결함	2026-04-23	35억 3,700만 원
한화호텔앤드리조트(주) · 권한 확인 누락 동일 결함	2024-08-28	1억 8,531만 원
현대자동차(주) · 보안 패치 미적용, 개인정보 노출 같은 유형	2024-08-28	329만 원

법정 상한은 최대치이며 예상 부과액이 아닙니다. 실제 부과액은 과징금 부과기준의 산정 단계(기준금액, 감경, 조정)를 거쳐 정해집니다.
유사 사례 금액이 상한보다 큰 것은 사례 기업의 매출 규모가 더 크기 때문이며, 같은 결함이 실제로 처분받은 규모를 보여주기 위한 참고값입니다.

7쪽 · 조치 로드맵 즉시 / 단기 / 중기

SK Shieldus Rookies

조치 로드맵

즉시 1주 이내

주석 내 민감 정보 마스킹 및 삭제
주석 내 정보 누출

단기 1개월 이내

비밀번호 복잡성 강화 및 검증 로직 추가
유추 가능한 비밀번호, 낮은 복잡도의 비밀번호 정책

중요 페이지에 추가 인증 절차 도입
불충분한 인증 절차

중기 분기 이내

문의글 접근 및 댓글 작성 권한 검증 추가
타인 문의글 열람, 타인 문의글 댓글 작성

비밀번호 복구 절차 강화 및 안전한 전달
취약한 비밀번호 복구 절차

8쪽 · 경영진 결정 사항 승인이 필요한 3건

SK Shieldus Rookies

경영진 결정이 필요한 사항

1. 강의 수강 정보 보호를 위해 주석 내 민감 정보 마스킹 작업을 즉시 승인해 주십시오.
2. 비밀번호 정책 강화를 위한 예산과 인력을 단기 내에 배정해 주십시오.
3. 보안 조치 및 투자 기록 관리를 위한 시스템을 도입해 주십시오.

취약점별 상세 내용과 조치 방법은 실무진용 상세 보고서(SCAN-20261007-002-T)에 있습니다. 법령·인증기준 매핑 중 24건은 아직 담당자 검토 전입니다.

04 보고서 생성 - 실무진용 보고서

⑨ 실무진용 상세 보고서 SCAN-20261007-002-T · A4 14쪽 — 판단 기준·조치 방법은 KISA 가이드 원문, 관련 법령은 조문 원문 그대로

3쪽 · 진단 결과 요약

위험도별 현황과 항목별 결과



모의 고육 플랫폼 취약점 진단 결과 | SCAN-20261007-002-T

SK Shieldus Rookies

2. 진단 결과 요약

점검 19개 항목 중 취약 7개, 양호 12개입니다. 전체 위험 수준은 **심각**(개인정보를 처리하는 자산에 '상' 위험 취약점이 있음)으로 판정했습니다.

2.1 위험도별 현황

위험도	취약 항목
상	7개
중	0개
하	0개

2.2 항목별 결과

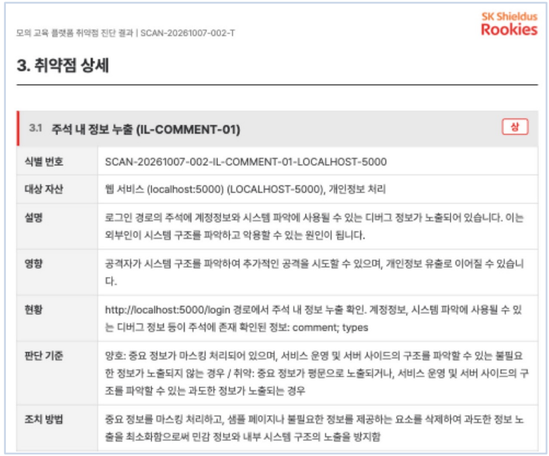
항목	점검 항목	대상 자산	위험도	결과
IL-COMMENT-01	주석 내 정보 누출	웹 서비스 (localhost:5000)	상	취약
BF-GUESS-01	유추 가능한 비밀번호	웹 서비스 (localhost:5000)	상	취약
BF-POLICY-01	낮은 복잡도의 비밀번호 정책	웹 서비스 (localhost:5000)	상	취약

코드가 판정하는 값

- 위험도별 취약 항목 수 (상 7 · 중 0 · 하 0)
- 항목별 취약 / 양호 — 점검 19개 중 취약 7개
- 전체 위험 수준 '심각' = 개인정보를 처리하는 자산에 '상' 위험 취약점이 있음
- LLM은 이 숫자를 직접 수정하지 않음

5쪽 · 취약점 상세 3.1

주석 내 정보 누출 (IL-COMMENT-01)



모의 고육 플랫폼 취약점 진단 결과 | SCAN-20261007-002-T

SK Shieldus Rookies

3. 취약점 상세

3.1 주석 내 정보 누출 (IL-COMMENT-01) 상

식별 번호	SCAN-20261007-002-IL-COMMENT-01-LOCALHOST-5000
대상 자산	웹 서비스 (localhost:5000) (LOCALHOST-5000), 개인정보 처리
설명	로그인 경로의 주석에 계정정보와 시스템 파악에 사용될 수 있는 디버그 정보가 노출되어 있습니다. 이는 외부인이 시스템 구조를 파악하고 악용할 수 있는 원인이 됩니다.
영향	공격자가 시스템 구조를 파악하여 추가적인 공격을 시도할 수 있으며, 개인정보 유출로 이어질 수 있습니다.
현황	http://localhost:5000/login 경로에서 주석 내 정보 누출 확인. 계정정보, 시스템 파악에 사용될 수 있는 디버그 정보 등이 주석에 존재 확인된 정보: comment; types
판단 기준	양호: 중요 정보가 마스킹 처리되어 있으며, 서비스 운영 및 서버 사이드의 구조를 파악할 수 있는 불필요한 정보가 노출되지 않는 경우 / 취약: 중요 정보가 평문으로 노출되거나, 서비스 운영 및 서버 사이드의 구조를 파악할 수 있는 과도한 정보가 노출되는 경우
조치 방법	중요 정보를 마스킹 처리하고, 샘플 페이지나 불필요한 정보를 제공하는 요소를 삭제하여 과도한 정보 노출을 최소화함으로써 민감 정보와 내부 시스템 구조의 노출을 방지함

한 항목에 들어가는 칸

- 식별 번호 · 대상 자산 — 진단 JSON 그대로
- 설명 · 영향 — LLM이 쓴 문장은 이 두 칸뿐
- 현황 — 진단 근거 원문 (확인된 정보까지)
- 판단 기준 · 조치 방법 — KISA 가이드 원문

9쪽 · 관련 법령 원문과 매핑 근거

조문 원문 + 매핑 근거 + '검토 전' 배지



모의 고육 플랫폼 취약점 진단 결과 | SCAN-20261007-002-T

SK Shieldus Rookies

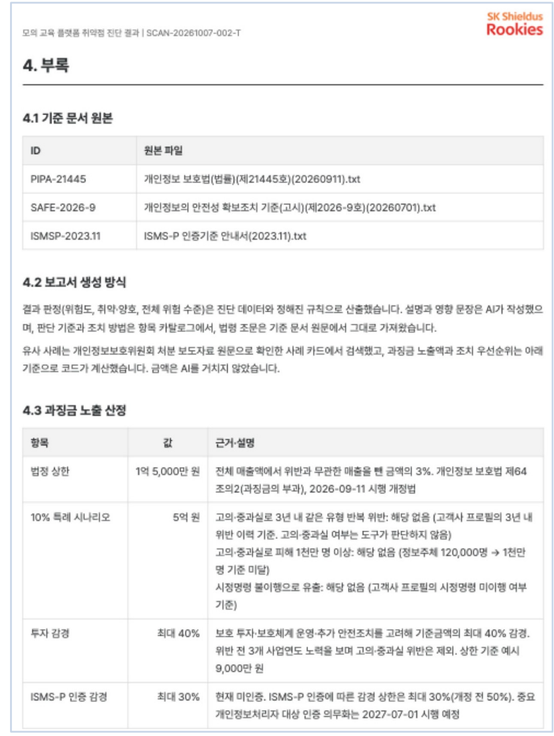
조치 방법	접근 제어가 필요한 모든 페이지에 서버 사이드 방식 사용자 권한 검증 로직 구현
관련 법령	**개인정보 보호법 제29조(안전조치의무)** 검토 전 제29조(안전조치의무) 개인정보처리자는 개인정보가 유출등이 되지 아니하도록 내부 관리계획 수립, 접속기록 보관 등 대통령령으로 정하는 바에 따라 안전성 확보에 필요한 기술적 · 관리적 및 물리적 조치를 하여야 한다. 매핑 근거: 타인 문의글 열람은 개인정보가 권한 없는 자에게 공개된 것으로, 개인정보 보호법 제29조의 안전성 확보 조치 의무를 위반한 사례임. (근거: ISMS-P 2.6.3(응용프로그램 접근) 관련 법규) **개인정보의 안전성 확보조치 기준 제5조 제5항(접근 권한의 관리)** 검토 전 ① 개인정보처리자는 개인정보처리시스템에 대한 접근 권한을 업무 수행에 필요한 최소한의 범위로 차등 부여하여야 한다. 매핑 근거: 타인 문의글 열람이 가능하다는 것은 업무 수행에 필요한 최소한의 범위로 접근 권한을 차등 부여하지 않은 것으로, 제1항의 접근 권한 최소 부여 의무 위반임. (근거: ISMS-P 2.6.3(응용프로그램 접근) 관련 법규) **개인정보의 안전성 확보조치 기준 제6조 제2항(접근통제)** 검토 전 ① 개인정보처리자는 처리하는 개인정보가 인터넷 홈페이지, P2P, 공유설정 등을 통하여 권한이 없는 자에게 공개되거나 유출되지 않도록 개인정보처리시스템, 개인정보취급자의 컴퓨터 및 모바일 기기 등에 조치를 하여야 한다. 매핑 근거: 권한 없는 자에게 개인정보가 공개된 점은 제3항의 권한 없는 자에게 개인정보 공개·유출 방지 조치 의무 위반임. (근거: ISMS-P 2.6.3(응용프로그램 접근) 관련 법규)
ISMS-P	**2.6.3 응용프로그램 접근** 검토 전 매핑 근거: 타인의 문의글이 URL 파라미터 변조로 열람 가능하여 응용프로그램 접근권한 제한 및 중요정보 노출 최소화 기준을 미충족함. **3.2.4 개인정보 목적 외 이용 및 제공** 검토 전 매핑 근거: 권한 없는 자가 타인의 개인정보를 조회할 수 있어 개인정보 목적 외 이용 및 제공 기준을 위반함.
매핑 검토 메모	LLM 항 판단으로 제외한 조문: SAFE-12 제12조(출력 · 복사시 안전조치): 진단 근거는 개인정보 출력·복사와 관련된 내용이 아니므로 해당하지 않음. / PIPA-18 제18조(개인정보의 목적 외 이용 · 제공 제한): 진단 근거에 개인정보 목적 외 이용 또는 제3자 제공에 관한 내용이 없어 해당하지 않음. / PIPA-19 제19조(개인정보를 제공받은 자의 이용 · 제공 제한): 진단 근거에 개인정보 제공받은 자의 이용·제공 제한 위반 관련 내용이 없어 해당하지 않음.

3.6 타인 문의글 댓글 작성 (IN-COMMENT-01) 상

식별 번호	SCAN-20261007-002-IN-COMMENT-01-LOCALHOST-5000
대상 자산	웹 서비스 (localhost:5000) (LOCALHOST-5000), 개인정보 처리
설명	타인의 문의글에 댓글을 작성할 수 있는 취약점이 존재합니다. 이는 권한 검증이 제대로 이루어지지 않은 결과입니다.
영향	권한 없는 자가 개인정보에 접근하여 악용할 수 있는 위험이 있습니다.

12쪽 · 부록 4.3 과징금 노출 산정

상한 · 특례 · 감경을 근거와 함께 표로



모의 고육 플랫폼 취약점 진단 결과 | SCAN-20261007-002-T

SK Shieldus Rookies

4. 부록

4.1 기준 문서 원본

ID	원본 파일
PIPA-21445	개인정보 보호법(법률)(제21445호)(20260911).txt
SAFE-2026-9	개인정보의 안전성 확보조치 기준(고시)(제2026-9호)(20260701).txt
ISMSP-2023.11	ISMS-P 인증기준 안내서(2023.11).txt

4.2 보고서 생성 방식

결과 판정(위험도, 취약·양호, 전체 위험 수준)은 진단 데이터와 정해진 규칙으로 산출했습니다. 설명과 영향 문장은 AI가 작성했으며, 판단 기준과 조치 방법은 항목 카탈로그에서, 법령 조문은 기준 문서 원문에서 그대로 가져왔습니다.

유사 사례는 개인정보보호위원회 처분 보도자료 원문으로 확인한 사례 카드에서 검색했고, 과징금 노출액과 조치 우선순위는 아래 기준으로 코드가 계산했습니다. 금액은 AI를 거치지 않았습니다.

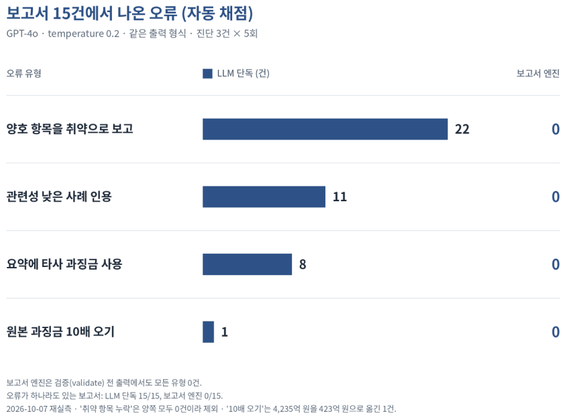
4.3 과징금 노출 산정

항목	값	근거·설명
법정 상한	1억 5,000만 원	전체 매출액에서 위반과 무관한 매출을 뺀 금액의 3%. 개인정보 보호법 제64조의2(과징금의 부과), 2026-09-11 시행 개정법
10% 특례 시나리오	5억 원	고의·중과실로 3년 내 같은 유형 반복 위반: 해당 없음 (고객사 프로필의 3년 내 위반 이력 기준. 고의·중과실 여부는 도구가 판단하지 않음) 고의·중과실로 피해 1천만 명 이상: 해당 없음 (정보주체 120,000명 → 1천만 명 기준 미달) 시정명령 불이행으로 유출: 해당 없음 (고객사 프로필의 시정명령 미이행 여부 기준)
투자 감경	최대 40%	보호 투자·보호체계 운영·추가 안전조치를 고려해 기준금액의 최대 40% 감경. 위반 전 3개 사업연도 노력을 보며 고의·중과실 위반은 제외. 상한 기준 예시 9,000만 원
ISMS-P 인증 감경	최대 30%	현재 미인증. ISMS-P 인증에 따른 감경 상한은 최대 30%(개정 전 50%). 중요 개인정보처리자 대상 인증 의무화는 2027-07-01 시행 예정

4.1 기준 문서 원본 목록 · 4.2 보고서 생성 방식 · 4.4 우선순위 점수와 공식 · 4.5 산정 가정과 제외 항목이 이어집니다.

04 보고서 생성 - 검증 결과

⑪ 비교 실험 LLM 단독 vs 보고서 엔진 — 진단 3건 × 5회, 총 30회 생성



실험 설계 두 조건 모두 GPT-4o · temperature 0.2 · 같은 JSON 출력 형식. 차이는 입력과 규칙뿐입니다. LLM 단독은 진단 JSON 원본, 매출이 들어간 고객사 정보, 과징금이 들어간 사례 카드를 그대로 받았고, 보고서 엔진은 실제 파이프라인 그대로 실행했습니다. 채점은 코드로 자동화했습니다. 다만 LLM 단독에는 사례 등급 정보를 주지 않았으므로, '관련성 낮은 사례' 차이는 등급 검색 설계의 효과까지 포함한 비교입니다.

오류 있는 보고서
LLM 단독
15/15
↓
보고서 엔진
0/15

양호 항목 오보
LLM 단독
22건
↓
보고서 엔진
0건

관련성 낮은 사례
LLM 단독
11건
↓
보고서 엔진
0건

요약 속 과징금
LLM 단독
8건
↓
보고서 엔진
0건

⑫ 정량 결과 모두 직접 실행해 측정한 값

0 건
엔진 오류
엔진 보고서 15건 오류 0건
(LLM 단독은 15건 모두 오류)

8/8
오류 탐지
가짜 ID · 금액 등 8종 주입 시험
8종 모두 탐지 · 제거 · 경고

10.5 초
보고서 1건
파이프라인 전체 실측 평균
최소 7.0 ~ 최대 16.0초

43%
입력 토큰 절감
LLM 단독 8,580 → 엔진 4,923
필요한 사실만 골라 전달

✓ **2026-10-07 재실측 (GPT-4o · 진단 3건 × 5회) · 요약 속 과징금 8건 = 15건 중 7건의 보고서 · 취약점 8건 중 7건에 동일 결함 사례 매칭 · 고객사 정보는 가상의 교육 플랫폼 기준**

에듀체크

05

생성형 AI를 활용한 개인정보 유출 취약점 자동 진단 및 ISMS-P 인증기준 확인

자체 평가

05 자체평가

박경민	보안 관련 프로젝트 경험이 적어 기능 구현이나 검증 부분에서 어려움이 있었지만, 취약점 진단 과정을 직접 경험하면서 적절한 검증 근거를 마련하고 제시할 수 있는 능력을 기를 수 있었던 것 같습니다.
송예린	LLM을 통한 자동화로 보고서 작업에 도움을 받더라도 검증 작업의 중요성을 놓치면 안되겠다는 생각을 했습니다. 카드를 추가하고 컨설팅 업체 맞춤형 작동을 위해 정보를 입력 받는 등 고도화하고 싶습니다.
어승효	짧은 기간으로 충분한 테스트를 진행하지 못한 점은 아쉬웠지만, HTTP 요청 기반 취약점 진단과 Git 통합을 경험하며 검증 과정과 협업 규격의 중요성을 배웠습니다.

05 자체평가

윤경은	일부 과정에 충분히 참여하지 못한 점은 아쉬웠지만, 통합 작업을 진행하며 전체 프로젝트 흐름을 이해할 수 있었습니다. 협업에서는 개별 기능 뿐 아니라 모듈 간 연결도 중요하다는 점을 배웠습니다.
장은빈	평소 지나치기 쉬운 비밀번호 찾기의 보안 취약성을 체감했습니다. 스캐너 구현이 낯설었지만 팀원들과 상의하며 취약점 분석과 협업의 중요성을 배웠습니다
최재용	안내서에 관련 법규가 없는 기준(101개 중 28개)은 조문을 비워 두어 해당 기준에 걸린 취약점은 법 조문 없이 사람이 직접 보완해야 함
홍용민	막연히 생성형 AI를 이용하면 취약점 진단을 자동화하기 쉬울 것이라 생각했는데, 고려해야할 사항이 많아 그렇지 않다는 것을 배웠습니다. 여러 플랫폼에 범용으로 사용 가능한 도구를 만들고자 하였으나, 만들어진 대상 서비스에 어느정도 종속적인 부분이 아쉬웠습니다.

감사합니다

Thank You